

What is C

- C is a general-purpose programming language created by Dennis Ritchie at Bell Laboratories in 1972.
- It is a very popular language, despite being old. The main reason for its popularity is that it is a fundamental language in computer science.
- C is strongly associated with UNIX, as it was developed to write the UNIX operating system.

The C Character set

- C Character set is a collection of characters supported in C programming language.
- The character set refers to a set of all the valid characters that we can use in the source program for forming words, expressions, and numbers.
- The C programming language provides support for the following types of characters. In other words, these are the *valid* characters that we can use in the C language:
 - **Alphabets**
 - **Digits**
 - **Special Symbols**

Why Learn C?

- It is one of the most popular programming languages in the world.
- If you know C, you will have no problem learning other popular languages like Java, Python, C++, and C#, as the syntax is similar.
- C is very fast compared to other programming languages like Java and Python.

The C Character set

Alphabets	A, B,, Y, Z a, b,, y, z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	~ ' ! @ # % ^ & * () _ - + = [] \ . , " ' < > ? /

What does C Character Set contains?

Alphabets

- C Language supports all alphabets of English. It supports both **UPPERCASE** & **lowercase** letters

Digits

- C Language supports 10 digits to construct numbers. Those 10 digits are **0,1,2,3,4,5,6,7,8,9**

Special Symbols

- - C supports a rich set of special symbols that include symbols to perform mathematical operations, condition checking, white space, back space, etc...

C Syntax

```
#include <stdio.h> // 1. Standard Input Output header
#include <conio.h> // 2. Console Input Output header

int main() { // 3. Main function - starting point of the program
    clrscr(); // 4. Clear the screen before printing output
    printf("Hello, World!\n"); // 5. Print Hello, World! to the console
    getch(); // 6. Wait for a key press to close the program
    return 0; // 7. End of the main function, returns 0 to the system
}
```

C Character Set

- In C Programming Language, the character set follow ASCII(American Standard Code for Information Interchange) code text format.
- Every character in C language has the respective ASCII value which is used to convert a character into Binary language.

Explanation of Each Part of the Program

1. #include <stdio.h>

- This line includes the Standard Input Output library.
- **Purpose:** Provides functions like printf() for outputting data to the console.

2. #include <conio.h>

- This line includes the Console Input Output library.
- **Purpose:** Provides functions like clrscr() and getch(), which are used to clear the screen and pause the program until a key is pressed.

Explanation of Each Part of the Program

3. `int main()`

- The main function is the entry point of any C program.
- **Purpose:** Code execution starts from the `main()` function. It returns an integer (int) value to the system.

4. `clrscr();`

- This function clears the screen, removing any text that might be present before the program starts.
- **Purpose:** To provide a clean screen before displaying new output. It's not a standard C function but is commonly used in Turbo C or similar environments.

How the Program Works

- **Start:** The program begins executing at `main()`.
- **Screen Clear:** `clrscr()` clears the console screen.
- **Output Display:** `printf()` prints "Hello, World!" on the console.
- **Pause:** `getch()` waits for a key press, preventing the console window from closing immediately.
- **End:** `return 0;` exits the program, indicating successful execution.

Explanation of Each Part of the Program

5. `printf("Hello, World!\n");`

- `printf` is used to display output on the screen.
- **Purpose:** Prints the text "Hello, World!" followed by a newline (`\n`) which moves the cursor to the next line.

6. `getch();`

- This function waits for the user to press a key before the program exits.
- **Purpose:** Used to pause the output screen so that the user can see the result before the console window closes.

7. `return 0;`

- Ends the `main()` function and returns 0 to the operating system.
- **Purpose:** `return 0;` indicates that the program has executed successfully.

Constant in C

C constants are fixed values that do not change during the execution of a program. Here are the main types of C constants:

• Integer Constants

- Represent whole numbers without a fractional part.
- Examples: 10, -5, 0
- Types: Decimal, Octal, Hexadecimal.

• Real (Floating-Point) Constants

- Represent numbers with a fractional part or exponential form.
- Examples: 3.14, -0.001, 6.022e23

Constant in C

3.Character Constants

- Represent a single character enclosed in single quotes.
- Examples: 'A', '9', '#'

4.String Constants

- Represent sequences of characters enclosed within double quotes.
- Examples: "Hello", "C Programming"

Rules for Constructing Integer Constants

1.Must Contain Digits Only:

- Integer constants consist of digits from 0 to 9.

2.No Decimal Point:

- Integer constants should not have a decimal point.

3.Positive or Negative:

- They can be preceded by a positive (+) or negative (-) sign, or no sign at all.

Rules for Constructing Integer Constants

4. Three Types of Integer Constants:

- **Decimal (Base 10):** Composed of digits from 0 to 9.
Examples: 123, -456
- **Octal (Base 8):** Begins with 0 and uses digits from 0 to 7.
Example: 012 (equivalent to decimal 10)
- **Hexadecimal (Base 16):** Begins with 0x or 0X and uses digits 0-9 and letters A-F.
Example: 0x1A (equivalent to decimal 26)

5. No Spaces or Special Characters:

- Integer constants cannot contain commas, spaces, or special symbols like \$.

Rules for Constructing Integer Constants

```
#include <stdio.h>
```

```
int main() {  
    int num1 = 123;    // Decimal constant  
    int num2 = -456;    // Negative decimal constant  
    int num3 = 051;    // Octal constant  
    int num4 = 0x1A;    // Hexadecimal constant  
  
    printf("Integer Constants: %d, %d, %d, %d\n", num1, num2,  
        num3, num4);  
    return 0;  
}
```

Rules for Constructing Real (Floating-Point) Constants

1. Must Contain a Decimal Point:

- Real constants must have a fractional part, i.e., a decimal point.
Example: 3.14

2. Can Be in Exponential Form:

- Real constants can also be written in exponential notation.
Example: 1.23e4 (equivalent to 1.23×10^4)

Rules for Constructing Real (Floating-Point) Constants

3. Can Be Positive or Negative:

- Real constants can be preceded by + or -.
Examples: -0.001, +23.45

4. Must Have At Least One Digit:

- At least one digit must be present before and after the decimal point in a real constant.
Example: 0.5

5. No Spaces or Special Characters:

- Real constants cannot contain spaces, commas, or symbols like #.

Rules for Constructing Real (Floating-Point) Constants

```
#include <stdio.h>
```

```
int main() {  
    float num1 = 3.14;    // Simple real constant  
    float num2 = -0.001;  // Negative real constant  
    float num3 = 2.5e3;   // Exponential form  
  
    printf("Real Constants: %.2f, %.3f, %.1f\n", num1,  
        num2, num3);  
    return 0;  
}
```

Rules for Constructing Character Constants

1. Enclosed in Single Quotes:

- Character constants must be enclosed within single quotes.
Example: 'A'

2. Single Character Only:

- Only one character, digit, or special symbol is allowed within the quotes.
Example: '7', '#'

Rules for Constructing Character Constants

3. Escape Sequences:

- Special characters can be represented using escape sequences starting with a backslash (\).
Examples: '\n' (newline), '\t' (tab), '\0' (null character)

4. Case Sensitivity:

- Character constants are case-sensitive, meaning 'A' and 'a' are different.

Rules for Constructing Character Constants

```
#include <stdio.h>
```

```
int main() {  
    char ch1 = 'A'; // Character constant  
    char ch2 = '9'; // Character constant representing a digit  
    char ch3 = '#'; // Special symbol as a character constant  
  
    printf("Character Constants: %c, %c, %c\n", ch1, ch2, ch3);  
    return 0;  
}
```

Variables in C

- In C language, variables are the storage place where some form of data is stored.
- Different variables require different amount of memory on which a set of operations is applied.
- A variable name cannot start with a number. It can consist of alphabets, number, underscore "_".

Variables in C

Here is the syntax of declaring variables in C language,

➤ type variable_name;

- Here is the syntax of multiple variables declaration in C language,

➤ Type variable_name1,
variable_name2, variable_name3;

Variables in C

Definition *Initialia*

↑

data_type variable_name = va

Variables in C

2. C Variable Definition

- In the definition of a C variable, the compiler allocates some memory and some value to it. A defined variable will contain some random garbage value till it is not initialized.

```
int var;
```

3. C Variable Initialization

- Initialization of a variable is the process where the user assigns some meaningful value to the variable when creating the variable.
- **Example**
- `int var = 10;`

Variables in C

There are 3 aspects of defining a variable:

- Variable Declaration
- Variable Definition
- Variable Initialization

1. C Variable Declaration

- Variable declaration in C tells the compiler about the existence of the variable with the given name and data type. When the variable is declared, an entry in symbol table is created and memory will be allocated at the time of initialization of the variable.

Types of C Variables

There are many types of variables in c:

- local variable
- global variable
- static variable etc

Types of Variables in C

1. Local Variables

- **Definition:** Variables declared inside a function or block. They are only accessible within that function or block.
- **Scope:** Limited to the function or block where they are declared.
- **Lifetime:** Exists only during the execution of the function or block.

Types of Variables in C

2. Global Variables

- **Definition:** Variables declared outside of all functions. They can be accessed by any function in the program.
- **Scope:** Throughout the entire program.
- **Lifetime:** Exists until the program ends.

Types of Variables in C

Example:

```
#include <stdio.h>
```

```
#include <conio.h> // For clrscr() and getch()
```

```
int main() {  
    clrscr(); // Clear the screen  
    int localVar = 10; // Local variable  
    printf("Local Variable: %d\n", localVar);  
    getch(); // Wait for user input before exiting  
    return 0;  
}
```

Explanation: localVar is a local variable that exists only inside the main function.

Types of Variables in C

```
#include <stdio.h>
```

```
#include <conio.h> // For clrscr() and getch()
```

```
int globalVar = 20; // Global variable
```

```
int main() {  
    clrscr(); // Clear the screen  
    printf("Global Variable: %d\n", globalVar);  
    getch(); // Wait for user input before exiting  
    return 0;  
}
```

Explanation: globalVar is accessible inside main because it is declared outside any function.

Types of Variables in C

3.Static Variables

- **Definition:** Variables declared with the static keyword. They retain their value between function calls.
- **Scope:** Local to the function or block but retain their value across multiple calls.
- **Lifetime:** Exists for the entire duration of the program.

Keywords in C

- Keywords are predefined, reserved words in C language and each of which is associated with specific features.
- These words help us to use the functionality of C language. They have special meaning to the compilers.
- There are total 32 keywords in C, Some of them are:-
- auto, double, int, struct, break, else, long, switch etc

Types of Variables in C

```
#include <stdio.h>
```

```
#include <conio.h> // For clrscr() and getch()
```

```
int main() {  
    clrscr();                // Clear the screen  
    static int staticVar = 30; // Static variable  
    printf("Static Variable: %d\n", staticVar);  
    staticVar++;             // Modifies staticVar for the next call (if any)  
    getch();                 // Wait for user input before exiting  
    return 0;  
}
```

Explanation: staticVar retains its value even after the function ends, which is seen if called multiple times.

Rules for constructing Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, price, carname, etc.).

Variable naming rules:

- A variable name must start with a letter or an underscore character (_)
- A variable name cannot start with a digit
- A variable name can only contain alpha-numeric characters and underscores (a-z, A-Z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- There is no limit on the length of the variable name
- A variable name cannot contain spaces
- The variable name cannot be any keywords

Rules for constructing Variable Names

Multi-Word Variable Names

- Variable names with more than one word can be difficult to read.
- There are several techniques you can use to make them more readable:

Camel Case

- Each word, except the first, starts with a capital letter:
- `myVariableName = "John"`

Pascal Case

- Each word starts with a capital letter:
- `MyVariableName = "John"`

Snake Case

- Each word is separated by an underscore character:
- `my_variable_name = "John"`

Comments in a C Program

Single-line Comments

- Single-line comments start with two forward slashes (`//`).
- Any text between `//` and the end of the line is ignored by the compiler (will not be executed).
- This example uses a single-line comment before a line of code:
- Example
- `// This is a comment`
`printf("Hello World!");`

Comments in a C Program

- Comments can be used to explain code, and to make it more readable. It can also be used to prevent execution when testing alternative code.
- Comments can be **singled-lined** or **multi-lined**.

Comments in a C Program

C Multi-line Comments

- Multi-line comments start with `/*` and ends with `*/`.
- Any text between `/*` and `*/` will be ignored by the compiler:

Example

- `/* The code below will print the words Hello World!`
to the screen, and it is amazing `*/`
`printf("Hello World!");`

Type Declaration Instruction

Type Declaration is a statement in programming that specifies the data type of a variable. Declaring the type of a variable helps the compiler to understand what kind of data the variable will hold, ensuring proper memory allocation and type safety during operations.

Key Points:

- **Purpose:** To inform the compiler about the variable type and its usage.
- **Syntax:** It varies between languages but generally follows the pattern:
 - **C/C++/Java:** type variableName; (e.g., int age;)

Type Conversion in Assignments

Type Conversion is the process of converting a variable from one data type to another. It happens in two ways:

1. Implicit Conversion (Automatic Type Conversion):

- The compiler automatically converts one data type to another without any explicit instruction from the programmer.
 - Occurs when a smaller data type is assigned to a larger data type
- ```
int a = 5;
float b = a; // Implicitly converts integer 'a' to float 'b'
```

## Examples:

### C/C++/Java:

- `int number;` - Declares a variable number of type integer.
- `float salary;` - Declares a variable salary of type float.

## Explicit Conversion (Type Casting):

The programmer explicitly defines the conversion using casting.

This is used when the conversion could result in data loss or when converting between incompatible types.

```
double x = 9.7;
```

```
int y = (int)x; // Explicitly converts double 'x'
 to integer 'y', truncating the decimal part
```

# Data Types in C language

Data types are fundamental aspects of programming that define the type of data a variable can hold. They determine how much memory is allocated to a variable and how the data is interpreted.

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc. Each data type requires different amounts of memory and has some specific operations which can be performed over it.

## The data types in C can be classified as follows:

| Types                   | Description                                                                                                                               | Data Types                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Primitive Data Types    | Primitive data types are the most basic data types that are used for representing simple values such as integers, float, characters, etc. | int, char, float, double  |
| Derived Types           | The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.                           | array, pointers, function |
| User Defined Data Types | The user-defined data types are defined by the user himself.                                                                              | structure, union, enum    |

## Basic Data Types in C

### Basic data types in C language

| Data Type          | Size in Bits | Range of values            |
|--------------------|--------------|----------------------------|
| Char               | 8            | -128 to +127               |
| unsigned char      | 8            | 0 to 255                   |
| signed char        | 8            | -128 to +127               |
| int                | 32           | -2147483648 to +2147483647 |
| signed int         | 32           | -2147483648 to +2147483647 |
| unsigned int       | 32           | 0 to 4294967295            |
| short              | 8            | -128 to +127               |
| short int          | 8            | -128 to +127               |
| signed short int   | 8            | -128 to +127               |
| unsigned short int | 8            | 0 to 255                   |

## 1. Integers

**Definition:** Integers are whole numbers without any fractional or decimal part. They are used to store numerical values such as counts, indexes, or any whole-number calculations.

### Types:

- **int:** Standard integer type.
- **Size:** Typically 2 or 4 bytes depending on the system, capable of holding values from -32,768 to 32,767 (for 2 bytes) or -2,147,483,648 to 2,147,483,647 (for 4 bytes).
- `int age = 25; //` Declares an integer variable 'age' with a value of 25

## 2. Long & Short Integers

### Long Integer (long):

- **Definition:** A longer version of the int type that can store larger numbers.
- **Size:** Typically 4 or 8 bytes.
- **Range:** Larger range than int, such as -2,147,483,648 to 2,147,483,647 (4 bytes) or beyond with 8 bytes.

`long population = 1000000000; // Declares a long integer for large numbers`

## 3. Signed & Unsigned Integers

### Signed Integer:

- **Definition:** Can hold both positive and negative values.
- **Example:**
- `int balance = -1500; // Can store negative numbers`

### Unsigned Integer (unsigned int):

- **Definition:** Can only hold non-negative numbers, doubling the positive range compared to signed integers.
- **Size:** Same as int, but the range is shifted to positive values only.
- **Example:**
- `unsigned int distance = 500; // Only stores positive values`

## Short Integer (short):

**Definition:** A shorter version of the int type that uses less memory.

- **Size:** Usually 2 bytes.
- **Range:** -32,768 to 32,767.
- **Example:**

`short score = 30000; // Suitable for smaller ranges`

## 4. Chars (Characters)

**Definition:** Used to store single characters or small integers. Each character is stored as an integer according to its ASCII value.

- **Size:** 1 byte.
- **Range:** Typically -128 to 127 (signed) or 0 to 255 (unsigned).
- **Types:**
- **Signed Char:** Can store negative and positive character values.
- **Unsigned Char:** Stores only positive values.

`char letter = 'A'; // Stores character 'A'`  
`signed char signedLetter = -65; // Stores negative ASCII values`  
`unsigned char age = 255; // Stores maximum unsigned value`

## 5. Float & Double

### Float:

- **Definition:** Used for storing single-precision floating-point numbers (decimals).
- **Size:** 4 bytes.
- **Range:** Approximately  $3.4E-38$  to  $3.4E+38$ .
- **Example:**
  - `float price = 19.99; // A variable with a fractional value`

## Double

**Definition:** Used for storing double-precision floating-point numbers, which allows more precision and a larger range.

- **Size:** 8 bytes.
- **Range:** Approximately  $1.7E-308$  to  $1.7E+308$ .
- **Example:**

`double preciseValue = 12345.678901234; //`  
Higher precision value

## Get the Memory Size

```
#include <stdio.h>
```

```
int main() {
 int myInt;
 float myFloat;
 double myDouble;
 char myChar;

 printf("%lu\n", sizeof(myInt));
 printf("%lu\n", sizeof(myFloat));
 printf("%lu\n", sizeof(myDouble));
 printf("%lu\n", sizeof(myChar));

 return 0;
}
```

**Note:** long unsigned int (%lu)

## Console Input/Output in C

- Console Input/Output (I/O) is a core feature of C programming that allows interaction between the user and the program through the console (command line or terminal). This interaction involves receiving inputs from the user and displaying outputs to the screen, enabling a basic form of user interaction.

## 1. Types of I/O in C

### Console I/O:

- Involves interacting with the console (terminal).
- Example: Reading user input via keyboard and displaying results on the screen.

### File I/O:

- Involves reading from or writing to files stored on disk.
- Example: Reading a text file for data or saving output results into a file.

### Network I/O:

- Involves sending and receiving data over networks.
- Example: Communicating with a web server using sockets (not commonly used in beginner-level C programming).

## 2. Console I/O Functions

Console I/O functions are used for direct interaction with the user. These functions can be broadly categorized into formatted and unformatted I/O function

## 3. Formatted Console I/O Functions

Formatted I/O functions in C allow control over how input is read and output is displayed. These functions use format specifiers to precisely control data presentation.

- **Key Functions:**

- **printf():**

- **Purpose:** Displays formatted output to the console.
- `printf("format string");`

- **Format Specifiers:**

`%d` – Integer, `%f` – Float, `%c` – Character, `%s` – String, `%lf` – Double, `%.nf` - Float with n decimal places

## 3. Formatted Console I/O Functions

### scanf():

- **Purpose:** Reads formatted input from the user.

`#include <stdio.h>`

```
int main() {
 int num;
 float price;
 printf("Enter a number and a price: ");

 scanf("%d %f", &num, &price); // %d for integer, %f for float
 printf("Number: %d, Price: %.2f\n", num, price); // Display
 with formatted output
 return 0;
}
```

**Note:** Use `&` to reference the variable addresses

## 4. Unformatted Console I/O Functions

Unformatted I/O functions handle input/output without using format specifiers, making them simpler but less flexible. They read or write data in a basic form without specific formatting.

### Key Functions:

- **getchar():**
  - **Purpose:** Reads a single character from the console.
- **putchar():**
  - **Purpose:** Writes a single character to the console.

### Example

```
#include <stdio.h>

int main() {
 char ch;
 char str[100]; // Array to store a string

 // Using getchar() and putchar()
 printf("Enter a character: ");
 ch = getchar(); // Reads a single character from input
 putchar(ch); // Prints the character to output
 printf("\n");

 // Using gets() and puts()
 printf("Enter a string: ");
 gets(str); // Reads a string from input
 puts(str); // Prints the string to output

 return 0;
}
```

## 4. Unformatted Console I/O Functions

- **gets()** (Unsafe, better to use fgets()):
  - **Purpose:** Reads a string from the console until a newline character is encountered.
- **puts():**
  - **Purpose:** Writes a string to the console, followed by a newline.

# PROBLEM SOLVING AND PROGRAMMING UNIT - 2

Prof. Prafful Raghaw

*Assistant Professor  
School of Computer Science Engineering  
Sandip University  
Sijoul, Madhubani  
Bihar*

# C Operators

Operators are used to perform operations on variables and values.

**C divides the operators into the following groups:**

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

## Example of Arithmetic Operator

```
#include <stdio.h>

int main() {
 int a = 10, b = 5;

 // Addition
 int sum = a + b;
 printf("Sum: %d\n", sum);

 // Subtraction
 int diff = a - b;
 printf("Difference: %d\n", diff);

 // Multiplication
 int product = a * b;
 printf("Product: %d\n", product);

 // Division
 int quotient = a / b;
 printf("Quotient: %d\n", quotient);

 // Modulus
 int remainder = a % b;
 printf("Remainder: %d\n", remainder);

 // Increment
 a++; // a = a + 1
 printf("Incremented a: %d\n", a);

 // Decrement
 b--; // b = b - 1
 printf("Decrement b: %d\n", b);

 return 0;
}

Output:
Sum: 15
Difference: 5
Product: 50
Quotient: 2
Remainder: 0
Incremented a: 11
Decrement b: 4
```

## Arithmetic Operators

- Arithmetic operators are used to perform common mathematical operations.

| Operator | Name           | Description                            | Example |
|----------|----------------|----------------------------------------|---------|
| +        | Addition       | Adds together two values               | x + y   |
| -        | Subtraction    | Subtracts one value from another       | x - y   |
| *        | Multiplication | Multiplies two values                  | x * y   |
| /        | Division       | Divides one value by another           | x / y   |
| %        | Modulus        | Returns the division remainder         | x % y   |
| ++       | Increment      | Increases the value of a variable by 1 | ++x     |
| --       | Decrement      | Decreases the value of a variable by 1 | --x     |

## Assignment Operators

- Assignment operators are used to assign values to variables

| Operator | Operation | Equivalent to |
|----------|-----------|---------------|
| =        | num = 5   | num = 5       |
| +=       | num+=5    | num = num+5   |
| -=       | num-=5    | num = num-5   |
| *=       | num*=5    | num = num*5   |
| /=       | num/=5    | num = num/5   |
| %=       | num%=5    | num = num%5   |

# Example of Assignment Operators

```
#include <stdio.h>

int main() {
 int a = 10, b = 5;

 // Assignment operator
 int result = a;
 printf("Result after '=': %d\n",
 result);

 // Add and assign
 result += b; // result = result + b
 printf("Result after '+=': %d\n",
 result);

 // Subtract and assign
 result -= b; // result = result - b
 printf("Result after '-=': %d\n",
 result);

 // Multiply and assign
 result *= b; // result = result * b
 printf("Result after '*=': %d\n",
 result);

 // Divide and assign
 result /= b; // result = result / b
 printf("Result after '/=': %d\n",
 result);

 // Modulus and assign
 result %= b; // result = result % b
 printf("Result after '%=': %d\n",
 result);

 return 0;
}
```

## Comparison Operators

- Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.
- The return value of a comparison is either 1 or 0, which means **true** (1) or **false** (0). These values are known as **Boolean values**

## Output

Result after '=': 10

Result after '+=': 15

Result after '-=': 10

Result after '\*=': 50

Result after '/=': 10

Result after '%=': 0

## Comparison Operators

| Operator | Name                     | Example | Description                                                                 |
|----------|--------------------------|---------|-----------------------------------------------------------------------------|
| ==       | Equal to                 | x == y  | Returns 1 if the values are equal                                           |
| !=       | Not equal                | x != y  | Returns 1 if the values are not equal                                       |
| >        | Greater than             | x > y   | Returns 1 if the first value is greater than the second value               |
| <        | Less than                | x < y   | Returns 1 if the first value is less than the second value                  |
| >=       | Greater than or equal to | x >= y  | Returns 1 if the first value is greater than, or equal to, the second value |
| <=       | Less than or equal to    | x <= y  | Returns 1 if the first value is less than, or equal to, the second value    |

## Example of Comparison Operators

```
#include <stdio.h>

int main() {
 int a = 10, b = 5;

 // Equal to
 printf("a == b: %d\n", a == b); // Will print 0 (false) because a is not equal to b

 // Not equal to
 printf("a != b: %d\n", a != b); // Will print 1 (true) because a is not equal to b

 // Greater than
 printf("a > b: %d\n", a > b); // Will print 1 (true) because a is greater than b

 // Less than
 printf("a < b: %d\n", a < b); // Will print 0 (false) because a is not less than b

 // Greater than or equal to
 printf("a >= b: %d\n", a >= b); // Will print 1 (true) because a is greater than or equal to b

 // Less than or equal to
 printf("a <= b: %d\n", a <= b); // Will print 0 (false) because a is not less than or equal to b

 return 0;
}
```

## Example of Logical Operators

```
#include <stdio.h>

int main() {
 int a = 10, b = 5, c = 0;

 // Logical AND (&&)
 printf("a > b && b > c: %d\n", (a > b && b > c)); // Will print 1 (true) because both
 conditions are true

 // Logical OR (||)
 printf("a > b || b < c: %d\n", (a > b || b < c)); // Will print 1 (true) because one
 condition is true

 // Logical NOT (!)
 printf("!(a == b): %d\n", !(a == b)); // Will print 1 (true) because a is not equal to b

 return 0;
}
```

## Logical Operators

Logical operators are used to determine the logic between variables or values, by combining multiple conditions

| Operator | Name | Example                    | Description                                      |
|----------|------|----------------------------|--------------------------------------------------|
| &&       | AND  | $x < 5 \ \&\& \ x < 10$    | Returns 1 if both statements are true            |
|          | OR   | $x < 5 \    \ x < 4$       | Returns 1 if one of the statements is true       |
| !        | NOT  | $!(x < 5 \ \&\& \ x < 10)$ | Reverse the result, returns 0 if the result is 1 |

## Bitwise Operators in C

- In C, the following 6 operators are bitwise operators (also known as bit operators as they work at the bit-level). They are used to perform bitwise operations in C.

## Bitwise Operators in C

- The **& (bitwise AND)** in C takes two numbers as operands and does AND on every bit of two numbers. The result of AND is 1 only if both bits are 1.
- The **| (bitwise OR)** in C takes two numbers as operands and does OR on every bit of two numbers. The result of OR is 1 if any of the two bits is 1.
- The **^ (bitwise XOR)** in C takes two numbers as operands and does XOR on every bit of two numbers. The result of XOR is 1 if the two bits are different.

### Truth table of the bitwise operators.

| X | Y | X & Y | X   Y | X ^ Y |
|---|---|-------|-------|-------|
| 0 | 0 | 0     | 0     | 0     |
| 0 | 1 | 0     | 1     | 1     |
| 1 | 0 | 0     | 1     | 1     |
| 1 | 1 | 1     | 1     | 0     |

## Bitwise Operators in C

- The **<< (left shift)** in C takes two numbers, the left shifts the bits of the first operand, and the second operand decides the number of places to shift.
- The **>> (right shift)** in C takes two numbers, right shifts the bits of the first operand, and the second operand decides the number of places to shift.
- The **~ (bitwise NOT)** in C takes one number and inverts all bits of it.

### Example of Bitwise Operators

```
// C Program to demonstrate use of bitwise operators
// The result is 00001100
printf("a^b = %u\n", a ^ b);

#include <stdio.h>
int main()
{
 // a = 5 (00000101 in 8-bit binary), b = 9 (00001001 in 8-bit binary)
 // 8-bit binary)
 unsigned int a = 5, b = 9;

 // The result is 00000001
 printf("a = %u, b = %u\n", a, b);
 printf("a&b = %u\n", a & b);

 // The result is 00001101
 printf("a|b = %u\n", a | b);

 // The result is 11111111111111111111111111111110
 // 10
 // (assuming 32-bit unsigned int)
 printf("~a = %u\n", a ~ a);

 // The result is 00010010
 printf("b<<1 = %u\n", b << 1);

 // The result is 00000100
 printf("b>>1 = %u\n", b >> 1);

 return 0;
}
```

## C Booleans

- C has a bool data type, which is known as **booleans**.
- Booleans represent values that are either true or false.
- In C, the bool type is not a built-in data type, like int or char.
- It was introduced in C99, and you must **import** the following header file to use it:
- `#include <stdbool.h>`
- you will need a data type that can only have one of two values, like:
- YES / NO
- ON / OFF
- TRUE / FALSE

## Example

```
#include <stdio.h>
#include <stdbool.h> // Import the boolean header file
```

```
int main() {
 bool isProgrammingFun = true;
 bool isFishTasty = false;
 printf("%d", isProgrammingFun); // Returns 1 (true)
 printf("%d", isFishTasty); // Returns 0 (false)

 return 0;
}
```

Output: 1 0

## C Booleans

- A **boolean** variable is declared with the **bool** keyword and can take the values true or false:
  - `bool a = true;`  
`bool abc = false;`
- Before trying to print the boolean variables, you should know that boolean values are returned as integers:**
- 1 (or any other number that is not 0) represents true
  - 0 represents false
  - Therefore, you must use the %d format specifier to print a boolean value:

## What is a Statement?

A **statement** in programming is a single line or instruction that performs an action. In C and many other languages, statements are executed sequentially, meaning one after another. Statements can include:

- Variable declarations (e.g., `int a = 5;`)
- Function calls (e.g., `printf("Hello, World!");`);)
- Control flow instructions like if statements, loops, etc.

# Statement in C

C has the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

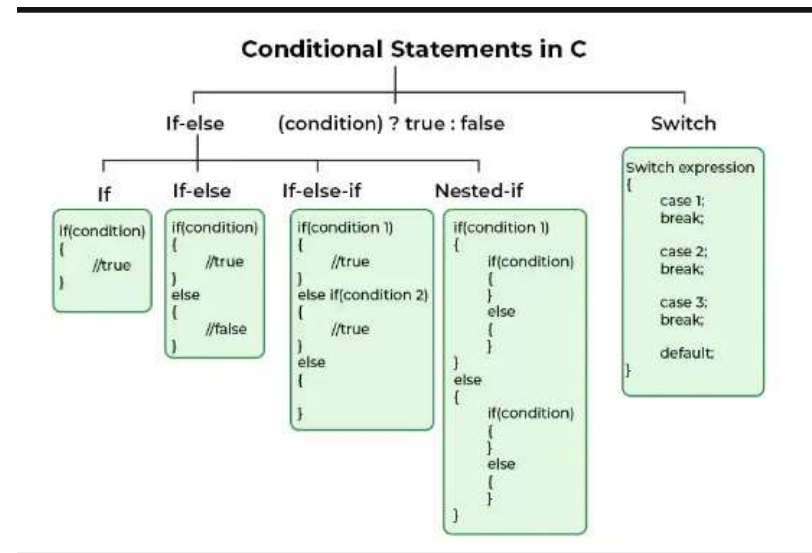
## 1. The if Statement

- The if statement allows the program to **execute a block of code** only when a certain **condition** is true. If the condition is false, the block of code is skipped.

**Syntax:**

```
if (condition) {
 // Code to execute if condition is true
}
```

# Statement in C



## 1. The if Statement

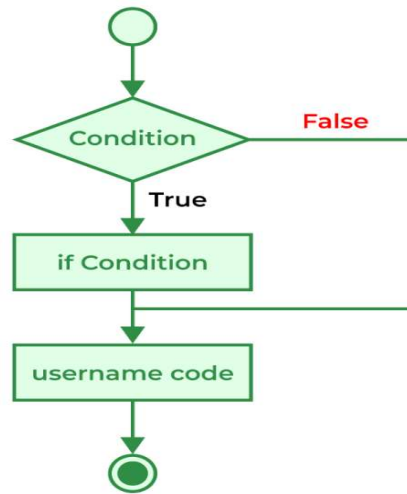
**Advantages:**

- **Simple to Understand:** The if statement is easy to read and understand, making it suitable for checking a single condition.
- **No Compulsion for Else:** You can check just one condition without needing to provide an else block.
- **Flexibility:** You can use multiple if statements independently if you need to check different conditions without a direct relationship.

**Disadvantages:**

- **Inefficiency for Multiple Conditions:** If multiple conditions need to be checked, using multiple if statements can lead to inefficient code, as each condition is checked even if one has already been true.
- **Repetitive Code:** In complex situations, multiple if statements can lead to code duplication if the conditions are related but treated separately.

## Flowchart of if Statement



## Example of if Statement

```
#include <stdio.h>
```

```
int main() {
 int age = 18;

 if (age >= 18) {
 printf("You are eligible to vote.\n");
 }

 return 0;
}
```

In this example, the message is printed because the condition `age >= 18` is true.

## Multiple Statements within if

- You can include multiple statements inside an if block by using curly braces `{}` to group them together. Without curly braces, only the first statement after the if is considered part of the conditional block.

### Syntax:

```
if (condition) {
 // Statement 1
 // Statement 2
 // ...
}
```

## Example of Multiple Statement within if

```
#include <stdio.h>
```

```
int main() {
 int score = 85;

 if (score > 80) {
 printf("Great job!\n");
 printf("You scored above 80.\n");
 }

 return 0;
}
```

Here, both print statements execute when the condition `score > 80` is true.

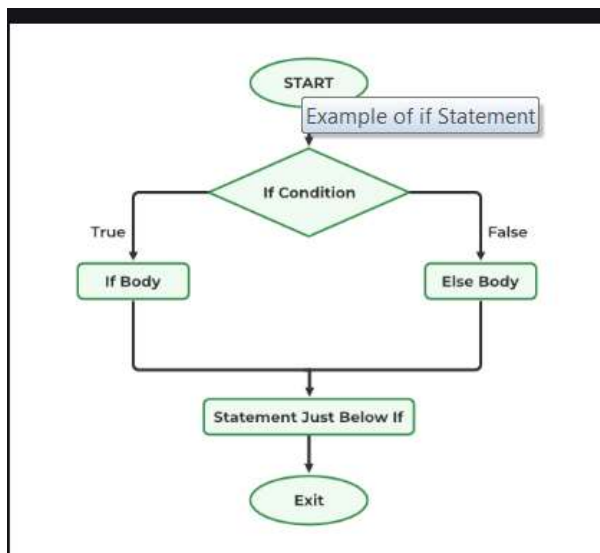
## 2. The if-else Statement

- The if-else statement allows the program to execute one block of code if the condition is true and another block if the condition is false.

### Syntax:

```
if (condition) {
 // Code to execute if condition is true
} else {
 // Code to execute if condition is false
}
```

## Flowchart of if-else Statement



## 2. The if-else Statement

### Advantages:

- **Dual Path:** The if-else statement allows for two possible outcomes (true or false), making the flow of logic clearer.
- **Efficiency:** It ensures that only one block of code is executed, making it efficient when there's a need to handle mutually exclusive conditions.
- **Clear Alternative:** It makes it explicit that if the if condition fails, the else block will execute, which simplifies the logic.

### Disadvantages:

- **Limited to Two Outcomes:** It is only useful when you have exactly two outcomes to handle. More complex decision-making requires chaining multiple conditions using else-if.
- **Less Flexible:** You cannot handle more than one alternate case, which limits its usefulness in scenarios with multiple conditions.

## Example of if-else Statement

```
#include <stdio.h>
```

```
int main() {
 int temperature = 25;

 if (temperature > 30) {
 printf("It's hot outside.\n");
 } else {
 printf("The weather is pleasant.\n");
 }

 return 0;
}
```

In this case, since temperature is not greater than 30, the second message ("The weather is pleasant.") will be printed.

## Nested if-else Statements

- You can place one if-else statement inside another if or else block. This is called **nesting**, and it allows you to check multiple conditions in a sequence.

### Syntax:

```
if (condition1) {
 // Code to execute if condition1 is true
} else {
 if (condition2) {
 // Code to execute if condition2 is true
 } else {
 // Code to execute if both conditions are false
 }
}
```

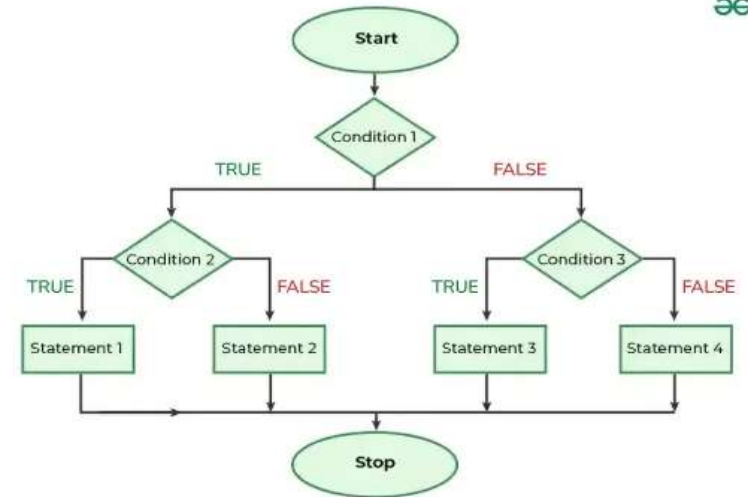
## Example of Nested if-else Statements

```
#include <stdio.h> class\n");
 }
int main() { }
 int marks = 65;

 if (marks >= 75) { }
 printf("Distinction\n");
 } else {
 if (marks >= 60) {
 printf("First
class\n");
 } else {
 printf("Second
```

Here, if marks >= 75, it prints "Distinction". If not, it checks if marks >= 60 and prints "First class". Otherwise, it prints "Second class".

## Flowchart of Nested if-else



## 3. The else if Statement

- Use the else if statement to specify a new condition if the first condition is false.

### Syntax

- if (condition1) {  
 // block of code to be executed if condition1 is true  
} else if (condition2) {  
 // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
 // block of code to be executed if the condition1 is false and condition2 is false  
}

### 3. The else if Statement

#### Advantages:

- **Handles Multiple Conditions:** The else-if ladder is great when you need to check multiple conditions in a prioritized manner. It allows you to specify different actions for each condition.
- **Efficient Execution:** Once a condition is met, the subsequent else-if conditions are not checked, making it more efficient for large decision structures.

#### Disadvantages:

- **Can Get Confusing:** If there are too many else-if statements, the code can become difficult to read and understand.
- **Nested Complexity:** Deeply nested if-else structures can make code hard to maintain and debug.

### Example of else if Statement

#### Example

```
#include <stdio.h>
```

```
int main() {
```

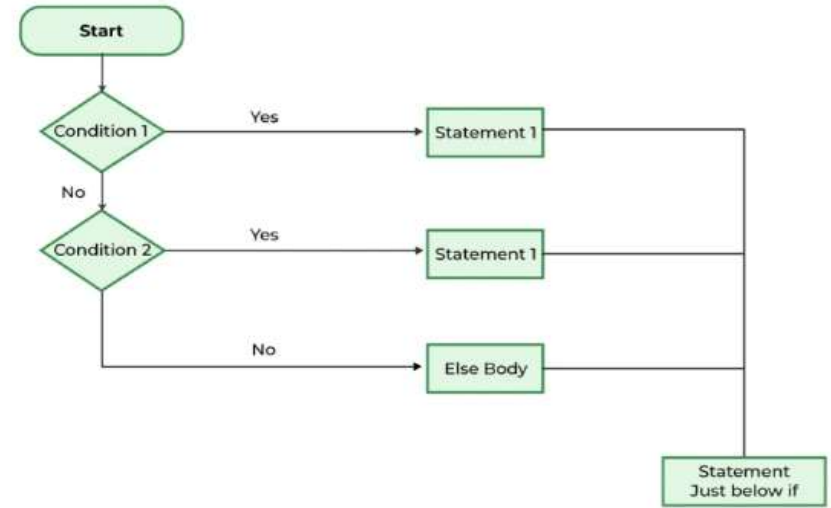
```
 int time = 22;
```

```
 if (time < 10) {
```

```
 printf("Good
morning.");
```

```
 } else if (time < 20) {
 printf("Good day.");
 } else {
 printf("Good
evening.");
 }
 return 0;
}
```

### Flowchart of if-else-if Ladder



### Summary of Key Points

| Feature     | if                        | if-else                   | else-if Ladder                 |
|-------------|---------------------------|---------------------------|--------------------------------|
| Use Case    | Simple condition checking | Two outcomes              | Multiple condition checks      |
| Efficiency  | Checks all conditions     | Checks one of two paths   | Stops checking after one match |
| Readability | Simple for one condition  | Clear for two cases       | Can get complex if overused    |
| Flexibility | No need for else block    | Handles binary cases well | Handles multiple scenarios     |

## The Conditional (Ternary) Operator

The **conditional operator** is a shorthand for if-else and is written as `? :`. It is used for simple conditions that return a value.

### Syntax:

`condition ? expression_if_true : expression_if_false;`

## Example of Conditional (Ternary) Operator

```
#include <stdio.h>
```

```
int main() {
 int a = 10, b = 20;

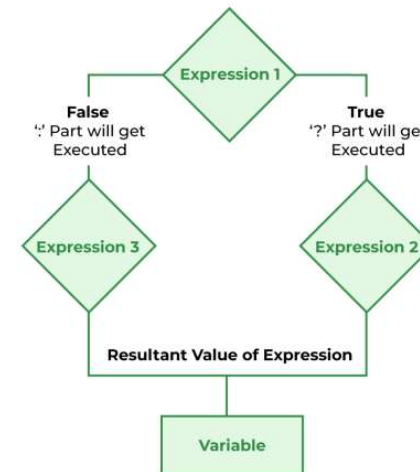
 int max = (a > b) ? a : b; // Find the maximum of a and b

 printf("The maximum value is: %d\n", max);

 return 0;
}
```

Here, the conditional operator checks if `a > b`. If true, it assigns `a` to `max`; otherwise, it assigns `b`. In this case, `b` is assigned since `a` is less than `b`.

## Flowchart of Conditional/Ternary Operator in C



## Decisions Using switch

- The switch statement in C is a control structure used to simplify the selection between different cases based on the value of a single variable or expression. It's a more efficient and readable alternative to multiple if-else conditions when dealing with a single variable compared to multiple constant values.
- **expression:** This is typically a variable or value that is evaluated once and compared with each case.
- **case:** Each case is a value that expression is compared against. If a match is found, the corresponding code is executed.
- **break:** It terminates the current case block to prevent the fall-through to the next case.
- **default:** A block that executes if none of the cases match the expression. This is optional but recommended.

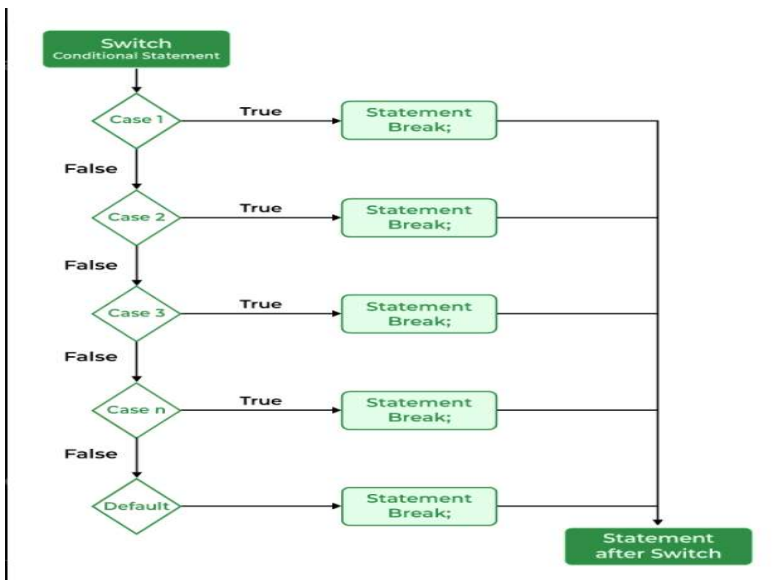
## Switch Statement in C

- Switch case statements follow a selection-control mechanism and allow a value to change control of execution.
- They are a substitute for long if statements that compare a variable to several integral values.
- The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

## Syntax of switch statement

```
switch (expression) {
 case constant1:
 // Code to execute if expression equals constant1
 break;
 case constant2:
 // Code to execute if expression equals constant2
 break;
 ...
 default:
 // Code to execute if no cases match
}
```

## Flowchart of switch



## Example of switch statement

```
#include <stdio.h>

int main() {
 int day = 3;

 switch (day) {
 case 1:
 printf("Monday\n");
 break;
 case 2:
 printf("Tuesday\n");
 break;
 case 3:
 printf("Wednesday\n");
 break;
 case 4:
 printf("Thursday\n");
 break;
 case 5:
 printf("Friday\n");
 break;
 default:
 printf("Invalid day\n");
 }

 return 0;
}
```

## Tips & Traps of Using switch

Using switch provides a clear way to handle multi-way branching, but there are some best practices and pitfalls to be aware of:

### Tips:

- **Use break Statements:** Always include a break after each case unless you intend for execution to "fall through" to the next case.
- **Always Use default Case:** This ensures that some code runs even if no case matches, making your program more robust.
- **Avoid Complex Expressions:** The switch statement works best with integers, characters or enums. Avoid using floating-point numbers or strings.

## switch Versus if-else Ladder

- The if-else ladder and switch are both conditional control structures in C, but they have different uses.

### When to Use switch:

- **Simple comparisons:** Use switch when you're checking a single variable against several constant values.
- **Readability:** The switch statement is more readable than an if-else ladder when there are many possible outcomes.

## Tips & Traps of Using switch

### Traps:

- **Forgetting the break Statement:** If you forget to include a break at the end of a case, execution will continue into the next case, which might lead to unexpected results.
- **Only Works with Discrete Values:** switch is not suitable for comparing ranges of values, complex conditions, or non-integer types.

## switch Versus if-else Ladder

### When to Use if-else:

- **Complex conditions:** Use if-else for checking ranges, relational conditions, or when evaluating multiple variables.
- **Non-integer comparisons:** Use if-else when working with floats, strings, or non-integer data types.

## Example switch Versus if-else Ladder

### Switch

```
int day = 2;
```

```
switch (day) {
 case 1:
 printf("Monday\n");
 break;
 case 2:
 printf("Tuesday\n");
 break;
 default:
 printf("Invalid\n");
}
```

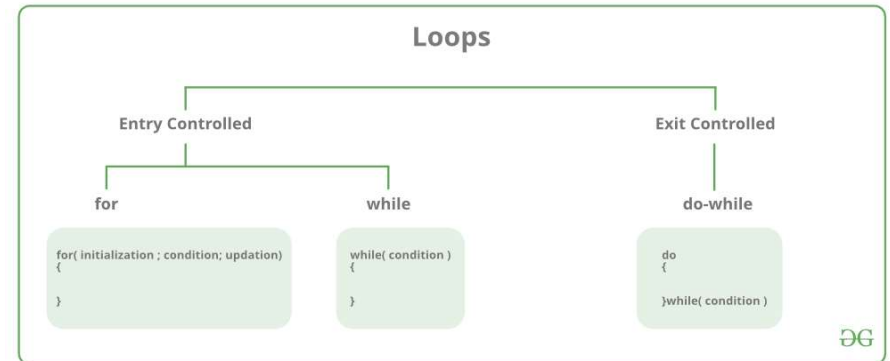
### If-else

```
int day = 2;
```

```
if (day == 1)
 printf("Monday\n");
else if (day == 2)
 printf("Tuesday\n");
else
 printf("Invalid\n");
```

## Loops

- A **loop** is a control structure that allows you to repeat a block of code multiple times as long as a specific condition is met. Loops help reduce redundancy and make your code more efficient.



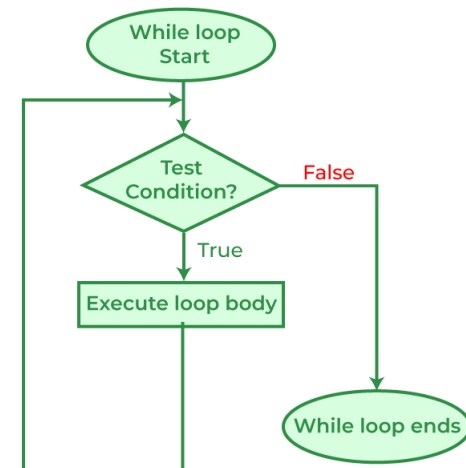
## The while Loop

- The while loop repeats a block of code as long as a condition is true. It checks the condition **before** executing the code, so if the condition is false at the start, the code inside the loop will not execute at all.

### Syntax:

```
while (condition) {
 // Code to execute as long as the condition is true
}
```

## Flow Diagram for while loop



## Example of while Loop

```
#include <stdio.h>
#include <conio.h>
```

```
int main() {
 int i = 1; // Variable declaration before the loop
 clrscr(); // Clear the screen

 while (i <= 5) {
 printf("%d\n", i);
 i++; // Increment the value of i
 }

 getch(); // Pause the output
 return 0;
}
```

**Explanation:** The loop will print numbers from 1 to 5, because the condition  $i \leq 5$  is true until  $i$  becomes 6.

## The for Loop

- The for loop is used when the number of repetitions is known. It includes initialization, condition, and increment/decrement in one line, making it compact and easy to use.

### Syntax:

```
for (initialization; condition;
 increment/decrement) { // Code to execute as
 long as the condition is true }
```

## while loop

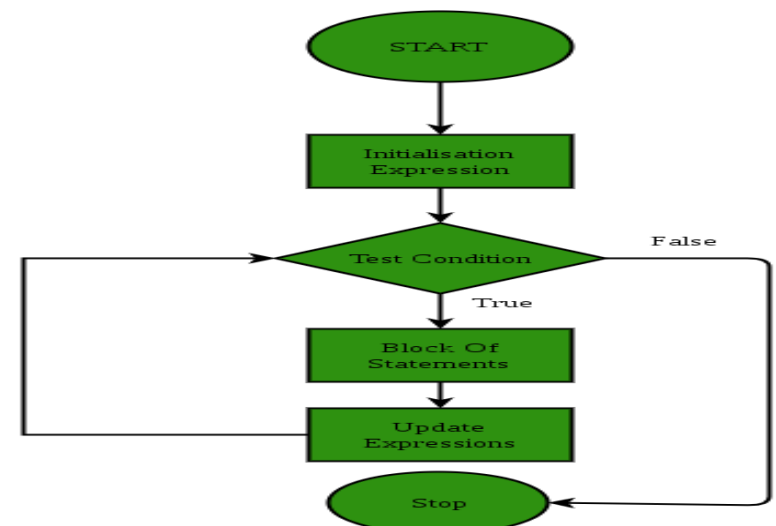
### Advantages:

- Simple and easy to use for cases where the number of iterations is unknown beforehand.
- Ideal for loops that depend on dynamic conditions.
- Can run zero times if the condition is false initially (provides flexibility).

### Disadvantages:

- Risk of infinite loops if the condition never becomes false.
- Condition checking happens before executing the loop body, so it might never execute if the condition is false from the start.
- Initialization, condition, and increment need to be written separately, which can increase code complexity.

## for loop Equivalent Flow Diagram



## Example of for Loop

```
#include <stdio.h>
#include <conio.h>
```

```
int main() {
 int i; // Declare the variable outside the loop
 clrscr(); // Clear the screen

 for (i = 1; i <= 5; i++) {
 printf("%d\n", i);
 }

 getch(); // Pause the output
 return 0;
}
```

**Explanation:** The loop prints numbers from 1 to 5. The variable i is declared outside the loop, which is necessary in Turbo C.

## Nesting of Loops

- Nesting means putting one loop inside another loop. In Turbo C, all loop control variables must be declared outside of the loops.

### Syntax:

```
for (initialization; condition; increment) {
 for (initialization; condition; increment) {
 // Code to execute inside the inner loop
 }
}
```

## for loop

### Advantages:

- Best suited for cases where the number of iterations is known beforehand.
- All loop control elements (initialization, condition, and increment) are in one line, making it more compact and readable.
- It reduces the risk of forgetting to update the loop variable (increment/decrement).

### Disadvantages:

- Can be less flexible compared to while loop when dealing with dynamic or complex loop conditions.
- Less intuitive for scenarios where iterations are based on conditions changing inside the loop.
- Can become confusing if initialization, condition, or increment involves complex logic.

## Example of Nesting of Loops

```
#include <stdio.h>
#include <conio.h>
```

```
int main() {
 int i, j; // Declare both loop variables outside
 clrscr(); // Clear the screen

 for (i = 1; i <= 3; i++) {
 for (j = 1; j <= 2; j++) {
 printf("i = %d, j = %d\n", i, j);
 }
 }

 getch(); // Pause the output
 return 0;
}
```

**Explanation:** The outer loop runs 3 times, and for each iteration of the outer loop, the inner loop runs 2 times.

## Multiple initialization inside for Loop in C

- We can have multiple initialization in the for loop as shown below.

```
for (i=1,j=1;i<10 && j<10; i++, j++)
```

**What's the difference between above for loop and a simple for loop?**

- It is initializing two variables. Note: both are separated by comma (,).
- It has two test conditions joined together using AND (&&) logical operator. Note: You cannot use multiple test conditions separated by comma, you must use logical operator such as && or || to join conditions.
- It has two variables in increment part. **Note:** Should be separated by comma.

## Example

```
#include <stdio.h>
int main()
{
 int i,j;
 for (i=1,j=1 ; i<3 || j<5; i++,j++)
 {
 printf("%d, %d\n",i ,j);
 }
 return 0;
}
```

## The break Statement

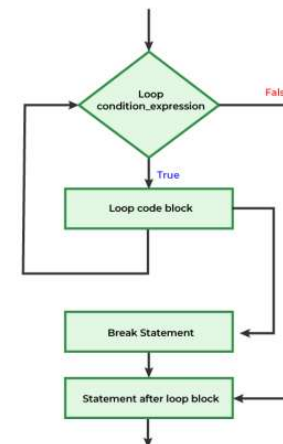
- The break statement is used to **immediately exit** a loop, regardless of whether the loop's condition is true or false.

### Syntax:

```
break;
```

## Flowchart of break in C

Break Statement Flow Diagram



## Example of break statement

```
#include <stdio.h>
```

```
int main() {
 for (int i = 1; i <= 5; i++) {
 if (i == 3) {
 break; // Exit the loop when i equals 3
 }
 printf("%d\n", i);
 }

 return 0;
}
```

**Explanation:** The loop will stop when i is equal to 3, so it will print 1 and 2 only.

## The continue Statement

The continue statement skips the current iteration of the loop and jumps to the next iteration.

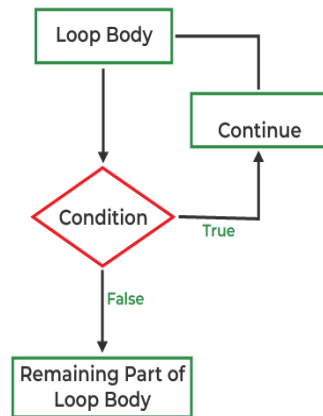
```
#include <stdio.h>
```

```
int main() {
 for (int i = 1; i <= 5; i++) {
 if (i == 3) {
 continue; // Skip the current iteration when i equals 3
 }
 printf("%d\n", i);
 }

 return 0;
}
```

**Explanation:** The loop will skip printing the number 3 and continue with the rest of the numbers.

## Flowchart of continue in C



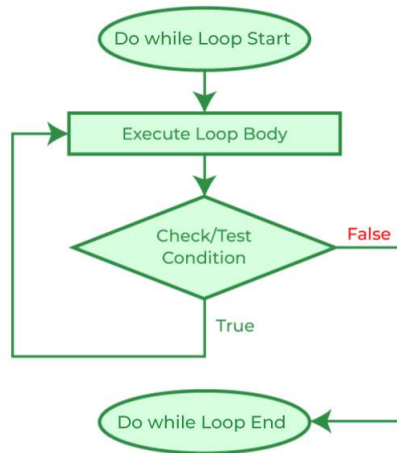
## The do-while Loop

- The do-while loop is similar to the while loop, but the **condition is checked after the code block executes**. This means that the code inside the loop will always execute at least once, even if the condition is false.

**Syntax:**

```
do {
 // Code to execute
} while (condition);
```

# do-while Loop Flow Diagram



## do-while loop

### Advantages:

- Guarantees that the loop body will execute **at least once** (useful for situations where the loop must run at least once).
- Useful in scenarios like user input validation, where the loop must execute before checking the condition.

### Disadvantages:

- Can lead to unintended behavior if you don't want the loop to run when the condition is false initially.
- Less commonly used, making it potentially less familiar to beginners.

## Example of do-while Loop

```
#include <stdio.h>
```

```
int main() {
 int i = 1;

 do {
 printf("%d\n", i);
 i++;
 } while (i <= 5);

 return 0;
}
```

**Explanation:** The loop will print numbers from 1 to 5. Even if *i* were initialized with a value greater than 5, the code inside the do block would still execute once before checking the condition.

## The goto Keyword

- The goto statement is used to jump to another part of the program. It allows you to move to a specific label, which is defined somewhere else in the code. While it is powerful, most programmers avoid using it because it can make programs messy and hard to follow. It can be useful in some rare situations, like breaking out of complicated loops or dealing with errors quickly.

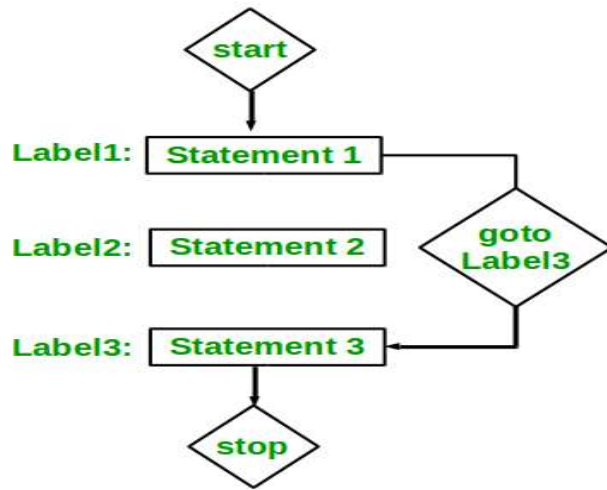
### What is goto?

- **Definition:** The goto statement tells the program to jump to a place marked by a label.
- **Label:** A label is simply a name followed by a colon (:) in your program where the control will jump.

### Syntax:

```
goto label; // This makes the program jump to the line with "label:"
label: // Code here will run when the program jumps to this label
```

## Flowchart of goto in C



## Example of goto Keyword

```
#include <stdio.h>
```

```
int main() {
 int x = 10;

 if (x == 10) {
 goto skip; // If x is 10, jump to the "skip" label
 }

 printf("This will be skipped\n"); // This line will be skipped

skip:
 printf("This is after the goto statement\n"); // Code at label "skip"

 return 0;
}
```

## Example of goto Keyword

### What happens?:

- The program checks if x is equal to 10. Since x is 10, it uses goto to jump to the label called skip.
- The line `printf("This will be skipped\n");` is **not** executed because the program jumps over it.
- It goes directly to the label skip: and prints "This is after the goto statement".

### Output:

- This is after the goto statement

# PROBLEM SOLVING AND PROGRAMMING UNIT - 3

Prof. Prafull Raghaw

*Assistant Professor  
School of Computer Science Engineering  
Sandip University  
Sijoul, Madhubani  
Bihar*

## Function in C

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- Functions are used to perform certain actions, and they are important for reusing code. Define the code once, and use it many times.

## How Functions Work

- In C, functions are used to organize code into reusable blocks. They are helpful for dividing complex programs into smaller, manageable parts. Functions in C follow a clear structure, which involves **declaration**, **definition**, and **calling**.

## Why Use Functions?

- **Code Reusability:** Write a function once, use it multiple times.
- **Modularity:** Programs are easier to understand when they are divided into functions.
- **Maintainability:** Updating code in a function changes all instances where the function is called.
- **Reduces Complexity:** Instead of one long sequence of instructions, functions allow you to manage small tasks separately.
- **Testing and Debugging:** Easier to test and debug.

## 1. Function Declaration

**Definition:** A function declaration tells the compiler about the function's name, return type, and parameters. It's also known as the function prototype.

### Syntax:

```
returnType functionName(parameterType1, parameterType2, ...);
```

- **returnType:** The data type of the value the function will return.
- **functionName:** The name of the function.
- **parameterType:** The type of parameters the function will take.

### Example:

- `int add(int , int ); // Function declaration`

## 2. Function Definition

**Definition:** A function definition provides the actual body of the function. It contains the logic that will be executed when the function is called.

**Syntax:**

```
returnType functionName(parameterType1
parameter1, parameterType2 parameter2, ...)
{ // Function body: statements to be executed
}
```

**Example:**

```
int add(int a, int b) {
 return a + b; // Function logic
}
```

## Example

```
#include <stdio.h>
#include <conio.h>

// Function declaration
int add(int a, int b);

int main() {
 clrscr(); // Clear the screen

 int x = 5, y = 10;
 int sum = add(x, y); // Calling
 // the function

 printf("Sum of %d and %d is
 %d\n", x, y, sum);

 getch(); // Wait for user input
 return 0;
}
```

## 3. Function Call

**Definition:** A function call is used to execute a function by using its name and passing required parameters (if any). The function is executed when it is called from the main() or another function.

**Syntax:**

```
functionName(argument1, argument2, ...);
```

**Example:**

```
int result = add(5, 10); // Calling the function
```

## Declare and Define Same Time

- It is possible to **define** and **declare** a function at the same time and then call it. When you define a function, you're also declaring it. This means that if you define the function before calling it in the code, there's no need for a separate function declaration.

## Example

```
#include <stdio.h>
#include <conio.h>

// Define and declare the function
// at the same time
int multiply(int a, int b) {
 return a * b; // Multiply two
 numbers and return the result
}

int main() {
 clrscr(); // Clear the screen

 int x = 4, y = 5;
 int result = multiply(x, y); // Call
 the function

 printf("Multiplication of %d and
 %d is %d\n", x, y, result);

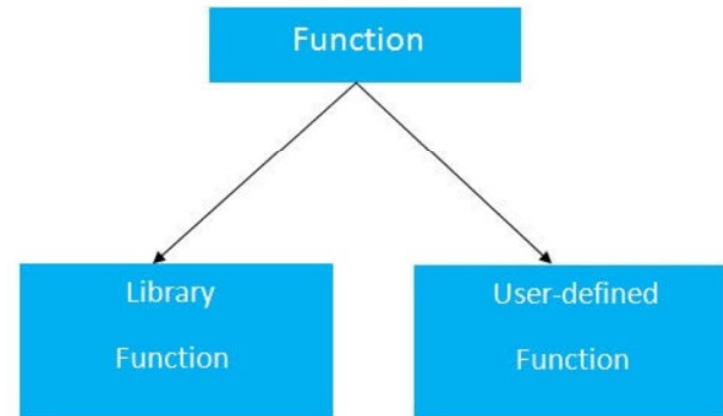
 getch(); // Wait for user input
 return 0;
}
```

## Types of Functions:

There are two types of function in C programming:

1. Standard Library Function
2. User Defined Function

## Types of Functions:



## Standard library functions

The standard library functions are built-in functions in C programming.

**These functions are defined in header files. For example,**

- The printf() is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in the stdio.h header file.

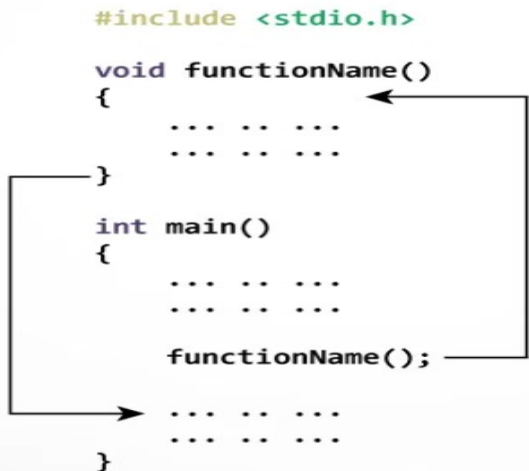
Hence, to use the printf() function, we need to include the stdio.h header file using #include <stdio.h>.

- The sqrt() function calculates the square root of a number. The function is defined in the math.h header file.

## User-defined function

- You can also create functions as per your need. Such functions created by the user are known as user-defined functions.
- The execution of a C program begins from the main() function.
- When the compiler encounters functionName();, control of the program jumps to void functionName().
- And, the compiler starts executing the codes inside functionName().
- The control of the program jumps back to the main() function once code inside the function definition is executed.

## How Function Works in C Programming



## Advantages of user-defined function

- The program will be easier to understand, maintain and debug.
- Reusable codes that can be used in other programs
- A large program can be divided into smaller modules. Hence, a large project can be divided among many programmers.

## Types of Functions Based on Return Type and Parameters

### 1. Function with No Return Type and No Parameters:

- This type of function performs a task but does not return any value or take any parameters.

#### Syntax:

- ```
void functionName() {
    // Code to execute
}
```

Function with No Return Type and No Parameters:

```
#include <stdio.h>
#include <conio.h>

void sayHello() {
    printf("Hello, World!\n");
}

int main() {
    clrscr();
    sayHello();
    getch();
    return 0;
}
```

2. Function with Parameters but No Return Type:

This type of function takes parameters but does not return any value.

Syntax:

```
void functionName(type parameter1, type
parameter2)
{
    // Code to execute
}
```

2. Function with Parameters but No Return Type:

Example:

```
#include <stdio.h>
#include <conio.h>

void add(int a, int b) {
    printf("Sum: %d\n", a + b);
}

int main() {
    clrscr();
    add(5, 10); // Passing arguments
    getch();
    return 0;
}
```

3. Function with Return Type but No Parameters:

This type of function returns a value but does not take any parameters.

Syntax:

```
return_type functionName()
{
    // Code to execute
    return value;
}
```

3. Function with Return Type but No Parameters:

Example:

```
#include <stdio.h>
#include <conio.h>
```

```
int getNumber() {
    return 10;
}
```

```
int main() {
    clrscr();
    int num = getNumber(); // Storing the return value
    printf("Number: %d\n", num);
    getch();
    return 0;
}
```

4. Function with Return Type and Parameters:

- This type of function takes parameters and returns a value.

Syntax:

```
return_type functionName(type parameter1,
                           type parameter2)
{
    // Code to execute
    return value;
}
```

4. Function with Return Type and Parameters:

Example:

```
#include <stdio.h>
#include <conio.h>
```

```
int multiply(int a, int b) {
    return a * b;
}
```

```
int main() {
    clrscr();
    int result = multiply(4, 5); // Passing arguments and storing return
    value
    printf("Product: %d\n", result);
    getch();
    return 0;
}
```

Scope Rule of Functions

The **scope** of a variable refers to where in the program the variable can be accessed. Variables can have **local** or **global** scope.

1. Local Scope:

- Variables declared inside a function are **local** to that function. They cannot be accessed outside the function.

Example of Local Scope Variable:

```
#include <stdio.h>
#include <conio.h>

void myFunction() {
    int localVar = 5; // Local variable
    printf("Local variable: %d\n", localVar);
}

int main() {
    clrscr();
    myFunction(); // Can access localVar only inside the function
    getch();
    return 0;
}
```

Order of Passing Arguments

- In C, arguments are passed to functions in the order they are listed. The first argument corresponds to the first parameter, and so on.
- **Note:** Always ensure that the number and type of arguments passed match the function's parameters.

2. Global Scope:

- Variables declared outside all functions are **global** and can be accessed from any function in the program.

```
#include <stdio.h>
#include <conio.h>

int globalVar = 10; // Global variable

void myFunction() {
    printf("Global variable: %d\n", globalVar); // Accessing global variable
}

int main() {
    clrscr();
    myFunction();
    getch();
    return 0;
}
```

Example

```
#include <stdio.h>
#include <conio.h>

void displayDetails(int age, float height) {
    printf("Age: %d, Height: %.2f\n", age, height);
}

int main() {
    clrscr();
    displayDetails(25, 5.9); // Passing arguments in correct order
    getch();
    return 0;
}
```

Library Functions

C provides several **library functions** (predefined functions) that can be used without needing to write them yourself. These are provided by header files like <stdio.h>, <math.h>, etc.

Common Library Functions:

- **printf()**: To print output to the screen.
- **scanf()**: To get input from the user.
- **strlen()**: To find the length of a string.
- **sqrt()**: To calculate the square root.

Examples of Library Functions

```
#include <stdio.h>
#include <conio.h> // For clrscr() and getch()
#include <string.h> // For strlen()

int main() {
    char str[50];
    int length;

    clrscr(); // Clear the screen

    printf("Enter a string: ");
    scanf("%s", str); // Using scanf to read a string (without spaces)

    length = strlen(str); // Finding the length of the string

    printf("Length of the string: %d\n", length); // Displaying the length

    getch(); // Wait for user input
    return 0;
}
```

Examples of Library Functions

```
#include <stdio.h>
#include <conio.h>
#include <math.h> // For sqrt()

int main() {
    clrscr();
    double num = 25.0;
    printf("Square root of 25: %.2f\n", sqrt(num)); // Using
    sqrt() function
    getch();
    return 0;
}
```

An Introduction to Pointers

- **Definition:** A pointer is a variable that stores the memory address of another variable. Pointers are powerful tools in C for dynamic memory management and for manipulating arrays and functions.

Syntax:

dataType *pointerName; // Declaration of a pointer

Why we use Pointers

- **Direct Memory Access:** Pointers allow you to access and manipulate memory directly by using the address of variables.
- **Dynamic Memory Allocation:** Pointers help allocate memory during program execution, useful for managing memory in data structures like arrays or linked lists.
- **Pass by Reference:** Pointers let you pass the address of variables to functions, so the function can modify the original variable.
- **Efficient Array Handling:** Pointers allow easy traversal and manipulation of arrays by pointing to their memory locations.
- **Memory Efficiency:** They enable efficient use of memory, especially when dealing with large data structures.

Example Explanation

- Create a pointer variable with the name `ptr`, that **points to** an int variable (`myAge`). Note that the type of the pointer has to match the type of the variable you're working with (int in our example).
- Use the `&` operator to store the memory address of the `myAge` variable, and assign it to the pointer.
- Now, `ptr` holds the value of `myAge`'s memory address.

Example of Pointers

```
#include <stdio.h>
int main() {
    int myAge = 43; // An int variable
    int* ptr = &myAge; // A pointer variable, with the name ptr, that stores the
                        // address of myAge

    // Output the value of myAge (43)
    printf("%d\n", myAge);

    // Output the memory address of myAge (0x7ffe5367e044)
    printf("%p\n", &myAge);

    // Output the memory address of myAge with the pointer (0x7ffe5367e044)
    printf("%p\n", ptr);
    return 0;
}
```

Dereference

- In the example above, we used the pointer variable to get the memory address of a variable (used together with the `&` **reference** operator).
- You can also get the value of the variable the pointer points to, by using the `*` operator (the **dereference** operator):

Example

```
#include <stdio.h>
```

```
int main() {  
    int myAge = 43; // Variable declaration  
    int* ptr = &myAge; // Pointer declaration  
  
    // Reference: Output the memory address of myAge with the  
    // pointer (0x7ffe5367e044)  
    printf("%p\n", ptr);  
  
    // Dereference: Output the value of myAge with the pointer (43)  
    printf("%d\n", *ptr);  
  
    return 0;  
}
```

Example of call by value

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void modifyValue(int num) {  
    num = num + 10; // Modifying the value  
}  
  
int main() {  
    clrscr(); // Clear the screen  
    int x = 5; // Original value  
    printf("Before function call: %d\n", x);  
    modifyValue(x); // Call by value  
    printf("After function call: %d\n", x); // x remains unchanged  
    getch(); // Wait for user input  
    return 0;  
}
```

Call by Value

- **Definition:** In Call by Value, a copy of the actual parameter's value is passed to the function. Changes made to the parameter inside the function do not affect the actual parameter.

Syntax:

```
void functionName(dataType parameter) {  
    // Function body  
}
```

Call by Reference

- **Definition:** In Call by Reference, instead of passing a copy of the actual parameter, a reference (or address) to the actual parameter is passed to the function. Changes made to the parameter inside the function affect the actual parameter.

Syntax:

```
void functionName(dataType *parameter) {  
    // Function body  
}
```

Example of Call by Reference

```
#include <stdio.h>
#include <conio.h>

void modifyValue(int *num) {
    *num = *num + 10; // Modifying the value at the address
}

int main() {
    clrscr(); // Clear the screen
    int x = 5; // Original value
    printf("Before function call: %d\n", x);
    modifyValue(&x); // Call by reference
    printf("After function call: %d\n", x); // x is modified
    getch(); // Wait for user input
    return 0;
}
```

Arrays

- An **array** is a collection of elements of the **same data type**, stored at **contiguous memory locations**. Arrays allow you to store multiple values using a **single variable name**.
- To create an array, define the data type (like int) and specify the name of the array followed by **square brackets []**.
- The elements are stored from left to right with the left-most index being the 0th index and the rightmost index being the (n-1) index.

Example:

- `int arr[5];` // Declares an array of 5 integers
To insert values to it, use a comma-separated list, inside curly braces:
`int myNumbers[] = {25, 50, 75, 100};`

Array Initialization

- **Array initialization** means assigning values to the elements of an array at the time of declaration.

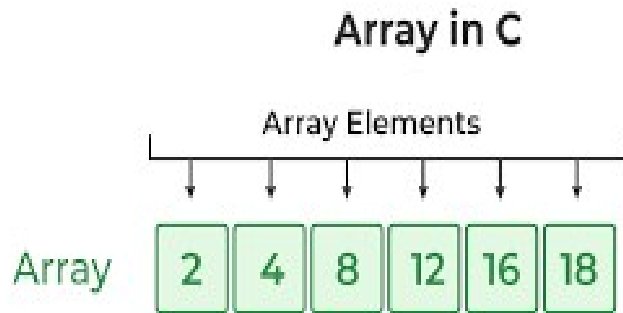
Syntax:

```
data_type array_name[size] = {value1, value2, ..., valueN};
```

Example:

```
int numbers[3] = {10, 20, 30}; // Initializes an array of 3 integers
```

C Arrays



Array Elements in Memory

- Array elements are stored in **contiguous memory locations**. Each element is placed next to the previous one in memory.
- For an array of integers, each element takes 4 bytes (on most systems), and they are stored back-to-back.

Example:

If you declare `int arr[3] = {1, 2, 3};`, the memory layout could look like this:

- `arr[0]` is at memory address 1000
- `arr[1]` is at memory address 1004
- `arr[2]` is at memory address 1008

Access the Elements of an Array

To access an array element, refer to its **index number**.

- Array indexes start with **0**: `[0]` is the first element. `[1]` is the second element, etc.
- This statement accesses the value of the **first element [0]** in `myNumbers`:

Example

- ```
int myNumbers[] = {25, 50, 75, 100};
printf("%d", myNumbers[0]);
```

## Change an Array Element

- To change the value of a specific element, refer to the index number:

### Example

```
myNumbers[0] = 33;
```

### Example

```
int myNumbers[] = {25, 50, 75, 100};
myNumbers[0] = 33;
```

```
printf("%d", myNumbers[0]);
```

```
// Now outputs 33 instead of 25
```

## Array Input/Output

```
#include <stdio.h>
int main() {
 int values[5];
 printf("Enter 5 integers: "); // taking input and storing it in an
 array
 for(int i = 0; i < 5; ++i) {
 scanf("%d", &values[i]);
 }
 printf("Displaying integers: "); // printing elements of the
 array
 for(int i = 0; i < 5; ++i) {
 printf("%d\n", values[i]);
 } return 0;
}
```

## Loop Through an Array

- You can loop through the array elements with the for loop.
- The following example outputs all elements in the myNumbers array:

### Example

```
int myNumbers[] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
 printf("%d\n", myNumbers[i]);
}
```

## Avoid Mixing Data Types

- It is important to note that all elements in an array **must be of the same data type**.
- This means you cannot mix different types of values, like integers and floating point numbers, in the same array:

### Example

- `int myArray[] = {25, 50, 75, 3.15, 5.99};`

## Making Better Loops

By using the sizeof formula from the example above, we can now make loops that work for arrays of any size, which is more sustainable.

### Instead of writing:

```
int myNumbers[] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
 printf("%d\n", myNumbers[i]);
}
```

### It is better to write:

```
int myNumbers[] = {25, 50, 75, 100};
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
int i;

for (i = 0; i < length; i++) {
 printf("%d\n", myNumbers[i]);
}
```

## Set Array Size

- Another common way to create arrays, is to specify the size of the array, and add elements later:

### Example

- `// Declare an array of four integers:`  
`int myNumbers[4];`

```
// Add elements
myNumbers[0] = 25;
myNumbers[1] = 50;
myNumbers[2] = 75;
myNumbers[3] = 100;
```

## A Simple Program using Array

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main() {
 clrscr();
 int arr[5] = {1, 2, 3, 4, 5}; // Array initialization

 for (int i = 0; i < 5; i++) {
 printf("Element at index %d: %d\n", i, arr[i]);
 }

 getch();
 return 0;
}
```

## Arrays Bounds Checking in C

- C does **not perform automatic bounds checking**. If you try to access an index outside the array size, it leads to **undefined behavior** (like accessing random memory locations).

### Example (Incorrect Access):

- `int arr[3] = {10, 20, 30}; printf("%d", arr[5]);` // Error: Accessing out of bounds, Gives some random values.

## Passing Array Elements to a Function

- You can pass **individual elements** of an array to a function just like passing any other variable. The function operates on a **copy** of the element.
- When you pass an element of the array, the function gets the value of that element, but changes inside the function do not affect the original array (because it's passed by value).

### Syntax:

- `void functionName(data_type element);`

## Example

```
#include <stdio.h>
```

```
void printElement(int element) {
 printf("Element: %d\n", element);
}

int main() {
 int arr[5] = {10, 20, 30, 40, 50};
 printElement(arr[2]); // Passing the third element (30) to the
 function
 return 0;
}
```

### Explanation:

- The function printElement takes one integer as an argument.
- We call the function with arr[2], which is the third element of the array.
- The value 30 is passed to the function, but the array itself is unaffected.

## Example

```
#include <stdio.h>
```

```
void printArray(int arr[], int size) {
 for (int i = 0; i < size; i++) {
 printf("%d ", arr[i]);
 }
 printf("\n");
}

int main() {
 int arr[5] = {1, 2, 3, 4, 5};
 printArray(arr, 5); // Passes the entire array to the
 function
 return 0;
}
```

## Passing the Whole Array to a Function

- In C, when you pass an array to a function, the array **name** acts as a pointer to the first element of the array. Therefore, changes made to the array inside the function will affect the original array.
- When passing the entire array, you need to pass the array name (without brackets) and the size of the array as arguments.

### Syntax:

```
void functionName(data_type array_name[], int size);
```

## Pointers and Arrays

### 1. What is an Array?

An array is a **collection of elements** of the same type stored in **contiguous memory locations**.

For example:

```
int arr[3] = {10, 20, 30};
```

Here, arr is an array of integers with 3 elements:

```
arr[0] = 10
```

```
arr[1] = 20
```

```
arr[2] = 30
```

# Pointers and Arrays

## . What is a Pointer?

A pointer is a variable that stores the **address** of another variable.

For example:c

```
int num = 10; int *ptr = # // 'ptr' now stores the address
```

# Relationship Between Arrays and Pointers

- The name of an array (e.g., arr) **acts as a pointer** to the first element of the array. arr is equivalent to &arr[0] (address of the first element).
- You can use a pointer to traverse and manipulate an array.

## Accessing Array Elements Using Pointers

You can access array elements using both:

**Array indexing** (arr[i])

**Pointer arithmetic** (\*(arr + i))

For example:

```
#include <stdio.h>
```

```
int main() {
 int arr[3] = {10, 20, 30};
```

```
 // Accessing elements using array indexing
```

```
 printf("arr[0] = %d\n", arr[0]);
```

```
 printf("arr[1] = %d\n", arr[1]);
```

```
 printf("arr[2] = %d\n", arr[2]);
```

```
 // Accessing elements using pointers
```

```
 printf("Using pointers:\n");
```

```
 printf("*arr + 0 = %d\n", *(arr + 0));
```

```
 printf("*arr + 1 = %d\n", *(arr + 1));
```

```
 printf("*arr + 2 = %d\n", *(arr + 2));
```

```
 return 0;
```

```
}
```

## Pointers and Arrays

```
#include <stdio.h>
```

```
int main() {
```

```
 int arr[4] = {10, 20, 30, 40};
```

```
 int *ptr = arr; // Pointer to the first element of the array
```

```
 printf("Accessing array elements using a pointer:\n");
```

```
 for (int i = 0; i < 4; i++) {
```

```
 printf("arr[%d] = %d, *(ptr + %d) = %d\n", i, arr[i], i, *(ptr + i));
```

```
 }
```

```
 return 0;
```

```
}
```

Output:

Accessing array elements using a pointer:

arr[0] = 10, \*(ptr + 0) = 10

arr[1] = 20, \*(ptr + 1) = 20

arr[2] = 30, \*(ptr + 2) = 30

arr[3] = 40, \*(ptr + 3) = 40

# Multidimensional Array

A **two-dimensional array** is like a table (matrix) with rows and columns where each element is identified by two indices: one for the row and one for the column.

## 1. Initializing a Two-Dimensional Array

A 2D array is declared as:

```
int arr[2][3]; // 2 rows and 3 columns
```

**You can initialize it directly:**

```
int arr[2][3] = { {1, 2, 3}, // Row 0 {4, 5, 6} // Row 1 };
```

**To access an element:**

`arr[0][1] = 2` (Row 0, Column 1)

`arr[1][2] = 6` (Row 1, Column 2)

# Memory Map of a Two-Dimensional Array

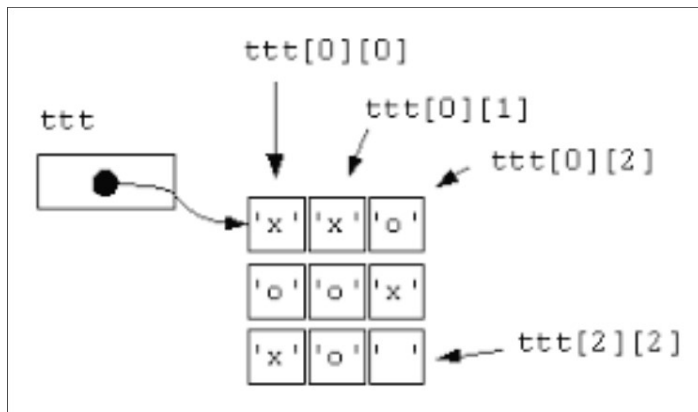
In memory, 2D arrays are stored **row by row (row-major order)**. Example: c

```
int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

**Memory layout:**

| Address | Value         |
|---------|---------------|
| 1000    | 1 (arr[0][0]) |
| 1004    | 2 (arr[0][1]) |
| 1008    | 3 (arr[0][2]) |
| 1012    | 4 (arr[1][0]) |
| 1016    | 5 (arr[1][1]) |
| 1020    | 6 (arr[1][2]) |

# Memory Map of a Two-Dimensional Array



# Pointers and Two-Dimensional Arrays

The name of a 2D array is a pointer to its first row.

`arr` points to `arr[0]` (address of the first row), and `arr[0]` points to the first element.

Example:

```
#include <stdio.h>
```

```
int main() {
 int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};

 printf("Address of arr: %p\n", arr);
 printf("Address of arr[0]: %p\n", arr[0]);
 printf("Value at arr[0][1]: %d\n", (*(arr + 0) + 1)); // Accessing arr[0][1]

 return 0;
}
```

Output:

Address of arr: 0x7ffeeabf34d0

Address of arr[0]: 0x7ffeeabf34d0

Value at arr[0][1]: 2

## Taking Input in 2D Array

```
#include <stdio.h>
void main ()
{
 int arr[3][3],i,j;
 for (i=0;i<3;i++)
 {
 for (j=0;j<3;j++)
 {
 printf("Enter a[%d][%d]: ",i,j);
 scanf("%d",&arr[i][j]);
 }
 }
}
```

```
printf("\n printing the elements .
...\n");
for(i=0;i<3;i++)
{
 printf("\n");
 for (j=0;j<3;j++)
 {
 printf("%d\t",arr[i][j]);
 }
}
```

## Array of Pointers

An array of pointers is a collection of pointers, where each pointer points to an array or a single variable.

Example:

```
#include <stdio.h>
```

```
int main() {
 int row1[] = {1, 2, 3};
 int row2[] = {4, 5, 6};
 int *arr[2] = {row1, row2}; // Array of pointers

 printf("Value at arr[0][1]: %d\n", arr[0][1]); // Accessing row1[1]
 printf("Value at arr[1][2]: %d\n", arr[1][2]); // Accessing row2[2]

 return 0;
}
```

Output:

Value at arr[0][1]: 2

Value at arr[1][2]: 6

## Passing 2D Array to a Function

You can pass a 2D array to a function by specifying the number of columns in the function parameter.

Example:

```
#include <stdio.h>
```

```
void printArray(int arr[2][3]) {
 for (int i = 0; i < 2; i++) {
 for (int j = 0; j < 3; j++) {
 printf("%d ", arr[i][j]);
 }
 printf("\n");
 }
}
```

```
}
int main() {
 int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
 printArray(arr);
}
```

return 0;

Output:

1 2 3

4 5 6

## Three-Dimensional Array

A **3D array** is like a box with multiple 2D tables.

Declaration:

```
int arr[2][3][4]; // 2 blocks, each with 3 rows and 4 columns
```

Example:

```
#include <stdio.h>
```

```
int main() {
 int arr[2][2][2] = {
 {{1, 2}, {3, 4}}, // Block 0
 {{5, 6}, {7, 8}} // Block 1
 };

 printf("Value at arr[1][0][1]: %d\n", arr[1][0][1]); // Accessing Block 1, Row 0, Column 1

 return 0;
}
```

Output:

Value at arr[1][0][1]: 6

## Strings in C

- Strings are used for storing text/characters.
- A **string** in C is a sequence of characters stored in a **character array** and terminated by a **null character (\0)**.
- For example, "Hello World" is a string of characters.
- Unlike many other programming languages, C does not have a **String type** to easily create string variables. Instead, you must use the char type and create an array of characters to make a string in C:
- `char greetings[] = "Hello World!";`

## Representation of Strings in Memory

- When you write a string like "Hello", it is stored in memory like this:  

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| H | E | L | L | O | \0 |
|---|---|---|---|---|----|
- The **null character (\0)** is automatically added at the end of the string to indicate where the string ends.

## Declaring Strings

You can declare strings in two ways:

**Using Character Array:**

```
char str[6] = "Hello"; // String with 6 characters (5 + 1 for '\0')
```

**Without Size:**

```
char str[] = "Hello";
```

**Character-by-Character:**

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

**Using Pointers:**

```
char *str = "Hello";
```

## More About Strings

- Strings are stored in memory as an array of characters. You can **input** and **output** strings using: `scanf()` and `printf()` (with `%s` format specifier).
- `gets()` and `puts()` (older functions, but not recommended in modern C).

## Input and Output of Strings

```
#include <stdio.h>
int main() {
 char str[20];
 printf("Enter a string: ");
 scanf("%s", str); // Input string without spaces printf("You
 entered: %s\n", str);
 return 0;
}
```

- **Note:** `scanf` stops reading a string when it encounters a space. To read a full sentence, you need `gets()` or `fgets()`.

## Strings in C

- Note that you have to use double quotes ("").
- To output the string, you can use the `printf()` function together with the format specifier `%s` to tell C that we are now working with strings:
- Example
- `char greetings[] = "Hello World!";  
printf("%s", greetings);`

## Access Strings

- Since strings are actually arrays in C, you can access a string by referring to its index number inside square brackets `[]`.
- This example prints the **first character (0)** in **greetings**:
- Example
- `char greetings[] = "Hello World!";  
printf("%c", greetings[0]);`
- Note that we have to use the `%c` format specifier to print a **single character**.

## Modify Strings

- To change the value of a specific character in a string, refer to the index number, and use **single quotes**:
- Example
- ```
char greetings[] = "Hello World!";  
greetings[0] = 'J';  
printf("%s", greetings);  
// Outputs Jello World! instead of Hello  
World!
```

Loop Through a String

- you can also use the *sizeof formula* (instead of manually write the size of the array in the loop condition ($i < 5$)) to make the loop more sustainable:

Example

- ```
char carName[] = "Volvo";
int length = sizeof(carName) / sizeof(carName[0]);
int i;

for (i = 0; i < length; ++i) {
 printf("%c\n", carName[i]);
}
```

## Loop Through a String

- You can also loop through the characters of a string, using a for loop:

Example

- ```
char carName[] = "Volvo";  
int i;  
  
for (i = 0; i < 5; ++i) {  
    printf("%c\n", carName[i]);  
}
```

Strings - Special Characters

- Because strings must be written within quotes, C will misunderstand this string, and generate an error:
- ```
char txt[] = "We are the so-called "Vikings" from the north.";
```
- The solution to avoid this problem, is to use the **backslash escape character**.
- The backslash (\) escape character turns special characters into string characters:

| Escape character | Result | Description  |
|------------------|--------|--------------|
| \'               | '      | Single quote |
| \"               | "      | Double quote |
| \\               | \      | Backslash    |

## Strings - Special Characters

The sequence \" inserts a double quote in a string:

```
char txt[] = "We are the so-called \"Vikings\" from the north.";
```

The sequence \' inserts a single quote in a string:

Example

```
char txt[] = "It\'s alright.";
```

The sequence \\ inserts a single backslash in a string:

Example

```
char txt[] = "The character \\ is called backslash.";
```

## Pointers and Strings

Strings can also be managed using **pointers**.

A **pointer** to a string points to the **first character** of the string.

**Example: Pointer to a String**

```
#include <stdio.h>
int main() {
 char *str = "Hello";
 printf("String is: %s\n", str); // Prints the entire string
 printf("First character: %c\n", *str); // Prints 'H'
 return 0;
}
```

**How It Works:**

char \*str = "Hello"; means str points to the first character 'H' of the string "Hello".

You can use **pointer arithmetic** to access other characters:

Copy code

```
*(str + 1) // Points to 'e'
```

## Standard Library String Functions

- The string.h library in C provides standard functions to work with strings:
- **strlen()**: Find the length of a string.
- **strcpy()**: Copy one string to another.
- **strcat()**: Concatenate (join) two strings.
- **strcmp()**: Compare two strings.

## String Functions

- C also has many useful string functions, which can be used to perform certain operations on strings.
- To use them, you must include the <string.h> header file in your program:
- #include <string.h>

**String Length**

- For example, to get the length of a string, you can use the strlen() function:
- Example
- char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
printf("%d", strlen(alphabet));

## String Functions

- We used sizeof to get the size of a string/array. Note that sizeof and strlen behaves differently, as sizeof also includes the \0 character when counting:
- Example
- ```
char alphabet[]  
= "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
printf("%d", strlen(alphabet)); // 26  
printf("%d", sizeof(alphabet)); // 27
```

Concatenate Strings

- To concatenate (combine) two strings, you can use the strcat() function:

Example

- ```
char str1[20] = "Hello ";
char str2[] = "World!";

// Concatenate str2 to str1 (result is stored in str1)
strcat(str1, str2);

// Print str1
printf("%s", str1);
```

## String Functions

- It is also important that you know that sizeof will always return the memory size (in bytes), and not the actual string length:
- Example
- ```
char alphabet[50]  
= "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
printf("%d", strlen(alphabet)); // 26  
printf("%d", sizeof(alphabet)); // 50
```

Copy Strings

- To copy the value of one string to another, you can use the strcpy() function:

Example

- ```
char str1[20] = "Hello World!";
char str2[20];

// Copy str1 to str2
strcpy(str2, str1);

// Print str2
printf("%s", str2);
```

## Compare Strings

- To compare two strings, you can use the strcmp() function.
- It returns 0 if the two strings are equal, otherwise a value that is not 0:
- Example
- ```
char str1[] = "Hello";  
char str2[] = "Hello";  
char str3[] = "Hi";
```

```
// Compare str1 and str2, and print the result  
printf("%d\n", strcmp(str1, str2)); // Returns 0 (the strings  
are equal)
```

```
// Compare str1 and str3, and print the result  
printf("%d\n", strcmp(str1, str3)); // Returns -4 (the strings  
are not equal)
```

Take String Input

- However, the scanf() function has some limitations: it considers space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words). For example:
- Example
- ```
char fullName[30];
```

```
printf("Type your full name: \n");
scanf("%s", &fullName);
```

```
printf("Hello %s", fullName);
```

```
// Type your full name: John Doe
// Hello John
```

## Take String Input

- You can also get a string entered by the user:
- Example
- Output the name of a user:
- ```
// Create a string  
char firstName[30];
```

```
// Ask the user to input some text  
printf("Enter your first name: \n");
```

```
// Get and save the text  
scanf("%s", firstName);
```

```
// Output the text  
printf("Hello %s", firstName);
```

Note: When working with strings in scanf(), you must specify the size of the string/array (we used a very high number, 30 in our example, but atleast then we are certain it will store enough characters for the first name), and you don't have to use the reference operator (&).

Take String Input

- From the example above, you would expect the program to print "John Doe", but it only prints "John".
- That's why, when working with strings, we often use the fgets() function to **read a line of text**. Note that you must include the following arguments: the name of the string variable, sizeof(string_name), and stdin:

Example

- ```
char fullName[30];
```

```
printf("Type your full name: \n");
fgets(fullName, sizeof(fullName), stdin);
```

```
printf("Hello %s", fullName);
```

```
// Type your full name: John Doe
// Hello John Doe
```

- Use the scanf() function to get a single word as input, and use fgets() for multiple words.

# Handling Multiple Strings

You can handle multiple strings using:

## 1. Two-Dimensional Array of Characters

```
char str[3][20] = {"Hello", "World", "C Programming"};
```

Here, str can hold 3 strings, each of size 20 characters.

## 2. Array of Pointers to Strings

```
char *str[] = {"Hello", "World", "C Programming"};
```

Each pointer points to a string.

# Multiple Strings

```
#include <stdio.h>
```

```
int main() {
 char str[3][20] = {"Hello", "World", "C Programming"};

 for (int i = 0; i < 3; i++) {
 printf("String %d: %s\n", i + 1, str[i]);
 }

 return 0;
}
```

### • Output:

- String 1: Hello
- String 2: World
- String 3: C Programming

# Array of Pointers to Strings

- An **array of pointers to strings** is a collection of **pointers**, where each pointer points to a **string** stored in memory.

```
#include <stdio.h>
```

```
int main() {
 char *str[] = {"Hello", "World", "C Programming"};

 for (int i = 0; i < 3; i++) {
 printf("String %d: %s\n", i + 1, str[i]);
 }

 return 0;
}
```

# Array of Pointers to Strings

## How It Works:

- str is an **array of pointers**.
- Each pointer (str[0], str[1], str[2]) points to a string literal.
- String literals ("Hello", "World", "C Programming") are stored in a special **read-only memory area**.

## Memory Representation:

- | str[0] → "Hello" | | str[1] → "World" | | str[2] → "C Programming" |

## Limitations of Array of Pointers to Strings

### 1. Strings Are Stored in Different Memory Locations

- Each string literal is stored in a separate memory location. Since the pointers in the array point to **different memory locations**, managing and modifying these strings can cause confusion.
- Why Is This a Problem?**
- You cannot perform bulk operations easily because each string is **not part of a continuous block of memory**.
- You must handle each pointer separately, which increases the complexity.

## Limitations of Array of Pointers to Strings

### 2. Strings Cannot Be Changed Easily Because They Are Read-Only

- When you use an array of pointers like `char *str[] = {"Hello", "World"};`, the strings "Hello" and "World" are stored as **string literals**.
- String literals are stored in a special **read-only section of memory**.

#### What Does "Read-Only" Mean?

- You **cannot modify** the characters of string literals.
- Attempting to change them will result in an **error** or undefined behavior.

## Limitations of Array of Pointers to Strings

```
#include <stdio.h>
```

```
int main() {
 char *str[] = {"Apple", "Banana", "Cherry"};

 printf("Address of str[0]: %p\n", str[0]);
 printf("Address of str[1]: %p\n", str[1]);
 printf("Address of str[2]: %p\n", str[2]);

 return 0;
}
```

**Output** (Memory addresses may vary):

Address of str[0]: 0x400804 Address of str[1]: 0x400810 Address of str[2]: 0x40081c

**Explanation:**

Each string "Apple", "Banana", and "Cherry" is stored in **different memory locations**. Even though they are logically part of an array (str), physically they are stored **far apart**.

You cannot perform operations that rely on strings being **contiguous in memory**.

## Limitations of Array of Pointers to Strings

```
#include <stdio.h>
```

```
int main() {
 char *str[] = {"Hello", "World"};

 str[0][0] = 'h'; // Trying to change 'H' to 'h'

 printf("%s\n", str[0]);

 return 0;
}
```

**Result:**

The program will **crash** or produce undefined behavior because "Hello" is stored in a **read-only memory section**.

# Limitations of Array of Pointers to Strings

## Why Does This Happen?

- str[0] points to the **string literal "Hello"**.
- String literals are stored in **read-only memory**.
- When you try to change a character (str[0][0] = 'h'), it violates the **read-only property**, causing the program to crash.

# Workaround to Modify Strings

- If you want to modify strings, you should use **character arrays** instead of pointers to string literals.
- ```
#include <stdio.h>
```

```
int main() {  
    char str[][10] = {"Hello", "World"};  
  
    str[0][0] = 'h'; // Changing 'H' to 'h'  
  
    printf("%s\n", str[0]); // Output: hello  
  
    return 0;  
}
```

Output:

hello

Explanation:

- Here, str is a **2D character array**.
- Strings are stored in a **modifiable area of memory**.
- You can safely change the characters.

Comparison: Array of Pointers vs. 2D Character Array

Feature	Array of Pointers	2D Character Array
Memory Storage	Strings are stored separately	Strings are stored contiguously
Modifiable	No (read-only literals)	Yes (modifiable)
Complexity	Higher complexity (pointers)	Simpler to manage
Usage	Used for fixed, read-only strings	Used for dynamic, modifiable strings

Structures in C

- A **structure** in C is a user-defined data type that allows grouping together different data types under one name. Think of it like a container that holds multiple variables of different types, all related to a specific concept.
- Each variable in the structure is known as a **member** of the structure.
- Use the struct keyword inside the main() method, followed by the name of the structure and then the name of the structure variable

Why Use Structures?

- In C, if you want to store multiple related pieces of data, you would typically use separate variables. But if the data is about a single entity, using separate variables can get messy. Structures solve this problem by grouping related variables together under one name.

Example Without Structures:

- If you're storing information about a student (e.g., name, roll number, and marks), you'd need:
- `char name[50];`
- `int rollNumber;`
- `float marks;`

Declaring a Structure

- You declare a structure using the `struct` keyword. Here's the syntax:

```
struct StructureName {  
    data_type member1;  
    data_type member2;  
    ...  
};
```

Example:

```
struct Student {  
    char name[50];  
    int rollNumber;  
    float marks;  
};
```

Why Use Structures?

- Here, managing these variables separately can be error-prone and inconvenient, especially when dealing with multiple students. Structures make it easier by grouping them into one unit.

Example With Structures:

```
struct Student {  
    char name[50];  
    int rollNumber;  
    float marks;  
};
```

- Now, you can manage all the details of a student as a single unit using the structure.

Declaring a Structure

- `struct` is the keyword to define a structure.
- `Student` is the name of the structure.
- `name`, `rollNumber`, and `marks` are the members of the structure.

Accessing Structure Elements

- To access elements of a structure, you use the **dot operator (.)**.

```
#include <stdio.h>

struct Student {
    char name[50];
    int rollNumber;
    float marks;
};

int main() {
    struct Student s1; // Declare a variable of type 'Student'

    // Assign values to structure members
    printf("Enter name: ");
    scanf("%s", s1.name);
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNumber);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    // Access and display structure members
    printf("\nStudent Details:\n");
    printf("Name: %s\n", s1.name);
    printf("Roll Number: %d\n", s1.rollNumber);
    printf("Marks: %.2f\n", s1.marks);

    return 0;
}
```

Explanation

- s1.name accesses the name member of s1.
- s1.rollNumber accesses the rollNumber member.
- s1.marks accesses the marks member.

What About Strings in Structures

- Remember that strings in C are actually an array of characters, and unfortunately, you can't assign a value to an array like this:

Example

```
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30]; // String
};

int main() {
    struct myStructure s1;

    // Trying to assign a value to the string
    s1.myString = "Some text";

    // Trying to print the value
    printf("My string: %s", s1.myString);

    return 0;
}
```

- An error will occur:
- prog.c:12:15: error: assignment to expression with array type
- However, there is a solution for this! You can use the strcpy() function and assign the value to s1.myString, like this:

Example

```
• struct myStructure {
    int myNum;
    char myLetter;
    char myString[30]; // String
};

int main() {
    struct myStructure s1;

    // Assign a value to the string using the strcpy function
    strcpy(s1.myString, "Some text");

    // Print the value
    printf("My string: %s", s1.myString);

    return 0;
}
```

Result:

- My string: Some text

Modify Values

- If you want to change/modify a value, you can use the dot syntax (.).
- And to modify a string value, the strcpy() function is useful again:

Example

```
• struct myStructure {
    int myNum;
    char myLetter;
    char myString[30];
};

int main() {
    // Create a structure variable and assign values to it
    struct myStructure s1 = {13, 'B', "Some text"};

    // Modify values
    s1.myNum = 30;
    s1.myLetter = 'C';
    strcpy(s1.myString, "Something else");

    // Print values
    printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

    return 0;
}
```

Copy Structures

- You can also assign one structure to another.
- In the following example, the values of s1 are copied to s2:

Example

- struct myStructure s1 = {13, 'B', "Some text"};
struct myStructure s2;

s2 = s1;

Example

```
• struct Car {
    char brand[50];
    char model[50];
    int year;
};

int main() {
    struct Car car1 = {"BMW", "X5", 1999};
    struct Car car2 = {"Ford", "Mustang", 1969};
    struct Car car3 = {"Toyota", "Corolla", 2011};

    printf("%s %s %d\n", car1.brand, car1.model, car1.year);
    printf("%s %s %d\n", car2.brand, car2.model, car2.year);
    printf("%s %s %d\n", car3.brand, car3.model, car3.year);

    return 0;
}
```

How Structure Elements are Stored?

- When a structure is declared, its elements are stored in **contiguous memory locations** in the order they are defined.

```
struct Example {
    int a;
    char b;
    float c;
};

int main() {
    struct Example e;
    printf("Size of structure: %lu\n", sizeof(e));
    return 0;
}
```

Array of Structures

- An **array of structures** allows you to store multiple records of the same structure type. This is useful for managing a collection of related data.

Explanation:

- The structure Example contains an int (4 bytes), a char (1 byte), and a float (4 bytes).
- Memory alignment (padding) may add extra space to ensure proper alignment for efficient access.

Example:

```
#include <stdio.h>

struct Student {
    char name[50];
    int rollNumber;
    float marks;
};

int main() {
    struct Student students[3]; // Array of 3
                                // structures

    // Input data for 3 students
    for (int i = 0; i < 3; i++) {
        printf("Enter details for student %d:\n", i
            + 1);
        printf("Enter name: ");
        scanf("%s", students[i].name);
        printf("Enter roll number: ");
        scanf("%d", &students[i].rollNumber);
        printf("Enter marks: ");
        scanf("%f", &students[i].marks);
    }

    // Display student details
    printf("\nStudent Details:\n");
    for (int i = 0; i < 3; i++) {
        printf("Student %d:\n", i + 1);
        printf("Name: %s\n", students[i].name);
        printf("Roll Number: %d\n",
            students[i].rollNumber);
        printf("Marks: %.2f\n",
            students[i].marks);
    }

    return 0;
}
```

Explanation:

- `struct Student students[3];` creates an array of 3 Student structures.
- Each element in the array represents a single student.
- You can access and manipulate individual elements using the array index (`students[0].name`, `students[1].rollNumber`, etc.).

Why And When To Use Enums

- Enums are used to give names to constants, which makes the code easier to read and maintain.
- Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.

C Enumeration (enum)

- An **enum** is a special type that represents a group of constants (unchangeable values).
- To create an enum, use the enum keyword, followed by the name of the enum, and separate the enum items with a comma:
- **enum Level {
 LOW,
 MEDIUM,
 HIGH
};**
- Note that the last item does not need a comma.
- It is not required to use uppercase, but often considered as good practice.
- Enum is short for "enumerations", which means "specifically listed".

Access enum

- To access the enum, you must create a variable of it.
- Inside the `main()` method, specify the enum keyword, followed by the name of the enum (Level) and then the name of the enum variable (`myVar` in this example):
- `enum Level myVar;`
- Now that you have created an enum variable (`myVar`), you can assign a value to it.
- The assigned value must be one of the items inside the enum (`LOW`, `MEDIUM` or `HIGH`):
- `enum Level myVar = MEDIUM;`
- By default, the first item (`LOW`) has the value 0, the second (`MEDIUM`) has the value 1, etc.
- If you now try to print `myVar`, it will output 1, which represents `MEDIUM`:

Example

```
int main() {  
    // Create an enum variable and assign a  
    value to it  
    enum Level myVar = MEDIUM;  
  
    // Print the enum variable  
    printf("%d", myVar);  
  
    return 0;  
}
```

Change Values

- Note that if you assign a value to one specific item, the next items will update their numbers accordingly:
- **enum Level {
 LOW = 5,
 MEDIUM, // Now 6
 HIGH // Now 7
};**

Change Values

- As you know, the first item of an enum has the value 0. The second has the value 1, and so on.
- To make more sense of the values, you can easily change them:
- **enum Level {
 LOW = 25,
 MEDIUM = 50,
 HIGH = 75
};**
- **printf("%d", myVar);** // Now outputs 50

Enum in a Switch Statement

- Enums are often used in switch statements to check for corresponding values:
- **enum Level {
 LOW = 1,
 MEDIUM,
 HIGH
};**

**int main() {
 enum Level myVar = MEDIUM;

 switch (myVar) {
 case 1:
 printf("Low Level");
 break;
 case 2:
 printf("Medium level");
 break;
 case 3:
 printf("High level");
 break;
 }
 return 0;
}**